COMSATS University Islamabad Wah Campus



Department of Computer Science

Internship Report (Week No. 5)

Submitted By :

Registration No : FA23-BAI-004

Name : Atif Ali Shah

Program : BSAI

Organization : Pakistan Ordnance Factory (POF) Wah Cantt (Rawalpindi)

Department : CO - 4 (IT)

Section : Database Administration (DBA)

Site Supervisor Name : Sir Babar Hussain **Signature :** _____

Submitted To :

Faculty Supervisor Name : Sir Ikram Ul Haq

Submission Date : 07 September 2026 (Monday)

Table of Contents

Chapter 7: Oracle Database Administration Using TOAD

Practical No. 9 — Database Link and Remote Database Access

7.1 Database Link and Remote Database Access
7.1.1 Introduction to Database Link
7.1.2 Purpose of Database Link
7.1.3 Local and Remote Database
7.1.4 Database Link Architecture
7.1.5 Requirements for Database Link
7.1.6 Creating a Database Link
7.1.7 Accessing Remote Database Tables
7.1.8 Verifying Database Link
7.1.9 Troubleshooting Common Database Link Errors
7.1.10 Dropping a Database Link

Practical No. 10 — Oracle Database Backup and Recovery

7.2 Database Backup and Recovery
7.2.1 Introduction to Database Backup
7.2.2 Importance of Database Backup
7.2.3 Types of Database Backup
7.2.4 Comparison of Backup Types
7.2.5 Physical Backup
7.2.6 Logical Backup
7.2.7 Cold Backup
7.2.8 Hot Backup
7.2.9 Full Backup
7.2.10 Incremental Backup
7.2.11 RMAN Introduction
7.2.12 RMAN Full Database Backup
7.2.13 RMAN Incremental Backup
7.2.14 Control File Backup
7.2.15 Archive Log Backup
7.2.16 Backup Verification
7.2.17 Restore and Recovery
7.2.18 RAC Introduction
7.2.19 RAC Architecture
7.2.20 RAC vs Single Instance

Practical No. 11 — Database Monitoring and Performance Tuning

7.3 Database Monitoring and Performance Tuning
7.3.1 Introduction to Database Monitoring
7.3.2 Monitoring Database Sessions
7.3.3 Checking Database Status
7.3.4 Checking Instance Status
7.3.5 Tablespace Monitoring
7.3.6 SQL Performance Monitoring
7.3.7 Execution Plan
7.3.8 Introduction to Performance Tuning
7.3.9 SQL Optimization
7.3.10 Indexes and Performance
7.3.11 AWR, ASH and ADDM Introduction

Practical No. 12 — Database Security and User/Privilege Management
7.4 Database Security and User Management
7.4.1 Introduction to Database Security
7.4.2 Database Users
7.4.3 Database Roles
7.4.4 System Privileges
7.4.5 Object Privileges
7.4.6 Creating a Database User
7.4.7 Granting Privileges
7.4.8 Revoking Privileges
7.4.9 Password Management
7.4.10 Principle of Least Privilege

7.5 Practical Outcomes

7.6 Conclusion

- 12.7 Practical Verification
- 12.8 Important Notes
- 12.9 Conclusion

Chapter 7: Oracle Database Administration Using TOAD

Practical No. 8: Database Link and Remote Database Access

7.1 Database Link and Remote Database Access

Objective

The objective of this practical is to understand and implement an Oracle Database Link for accessing data from a remote Oracle database.

A Database Link allows one Oracle database to access objects, such as tables and views, that are located in another Oracle database.

This practical demonstrates how a DBA can establish communication between databases located on different computers.

7.1.1 Introduction to Database Link

A **Database Link** is a database object in Oracle that provides a connection from one Oracle database to another Oracle database.

Once a Database Link is created, a user can access objects in the remote database using the database link name.

For example:

```
SELECT *
FROM employees@remote_db;
```

Here:

- `employees` is the remote table.
- `remote_db` is the Database Link name.

7.1.2 Purpose of Database Link

Database Links are useful for:

- Accessing data from another Oracle database.
- Sharing information between databases.
- Distributed database environments.
- Data integration.
- Remote queries.
- Database administration and testing.

7.1.3 Local and Remote Database

In a Database Link configuration, two databases are involved:

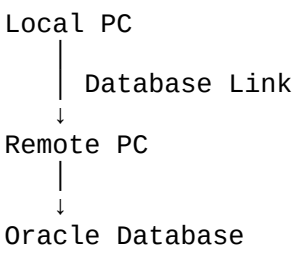
Local Database

The database from which the query is executed.

Remote Database

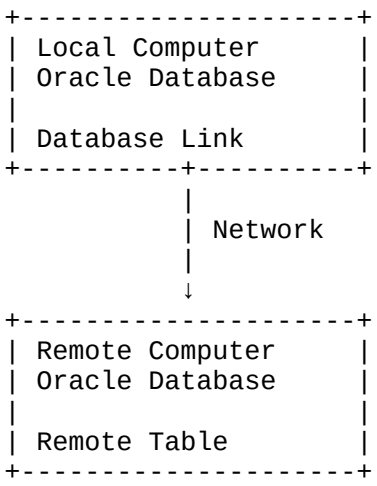
The database that contains the table or object being accessed.

Example:



7.1.4 Database Link Architecture

The basic architecture is:



The local Oracle database sends the request through the Database Link to the remote Oracle database.

7.1.5 Requirements for Database Link

The following requirements should be fulfilled:

1. Both computers should be connected through a network.
2. The remote Oracle listener should be running.
3. The remote database service should be available.
4. The required port, normally `1521`, should be accessible.
5. The remote database should have a valid user.
6. The TNS/service information should be correctly configured where applicable.
7. The user creating the Database Link should have the required privilege.

7.1.6 Creating a Database Link

After verifying connectivity, connect to the local Oracle database using the appropriate user.

The general syntax is:

```
CREATE DATABASE LINK remote_db
CONNECT TO username
IDENTIFIED BY password
USING 'service_name';
```

Example:

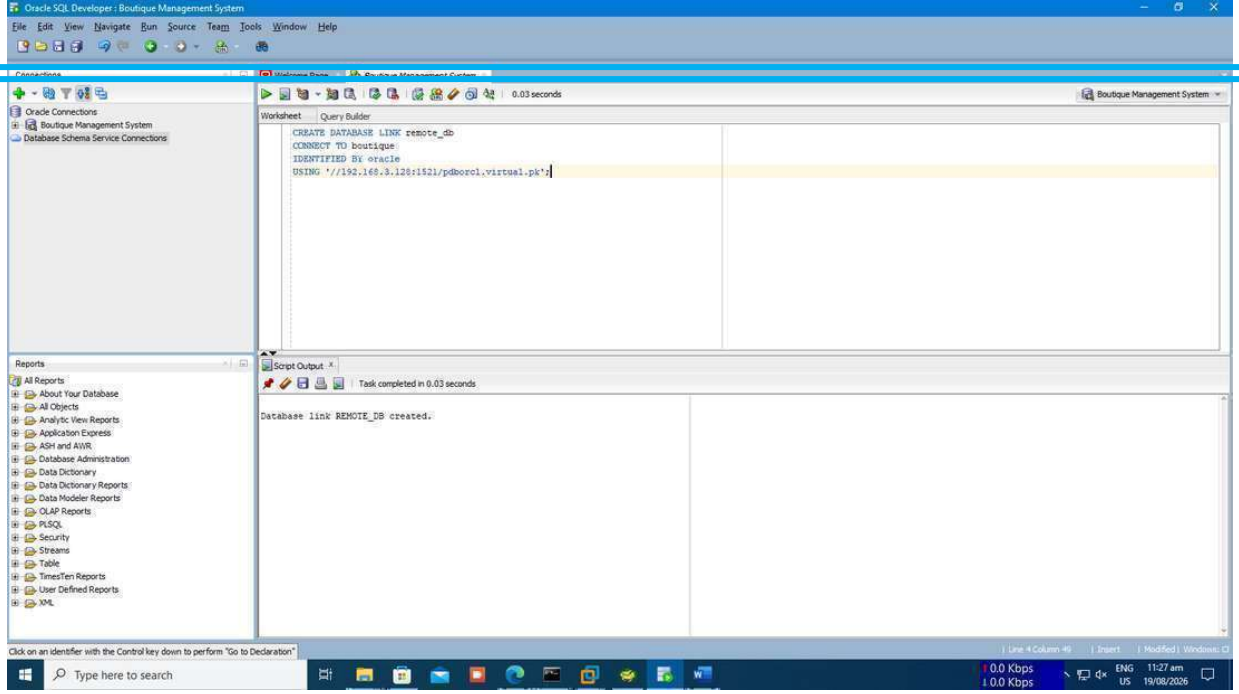
CREATE DATABASE LINK remote_db
CONNECT TO boutique
IDENTIFIED BY oracle
USING '//192.168.3.128:1521/pdborcl.virtual.pk';

Important: The actual username, password, host, service name and Database Link name will depend on the two Oracle databases available in your lab.

Explanation

The `CREATE DATABASE LINK` statement creates a database object that allows the local database to communicate with the remote Oracle database.

Screenshot 7.2: Creating Database Link



7.1.7 Accessing a Remote Database Table

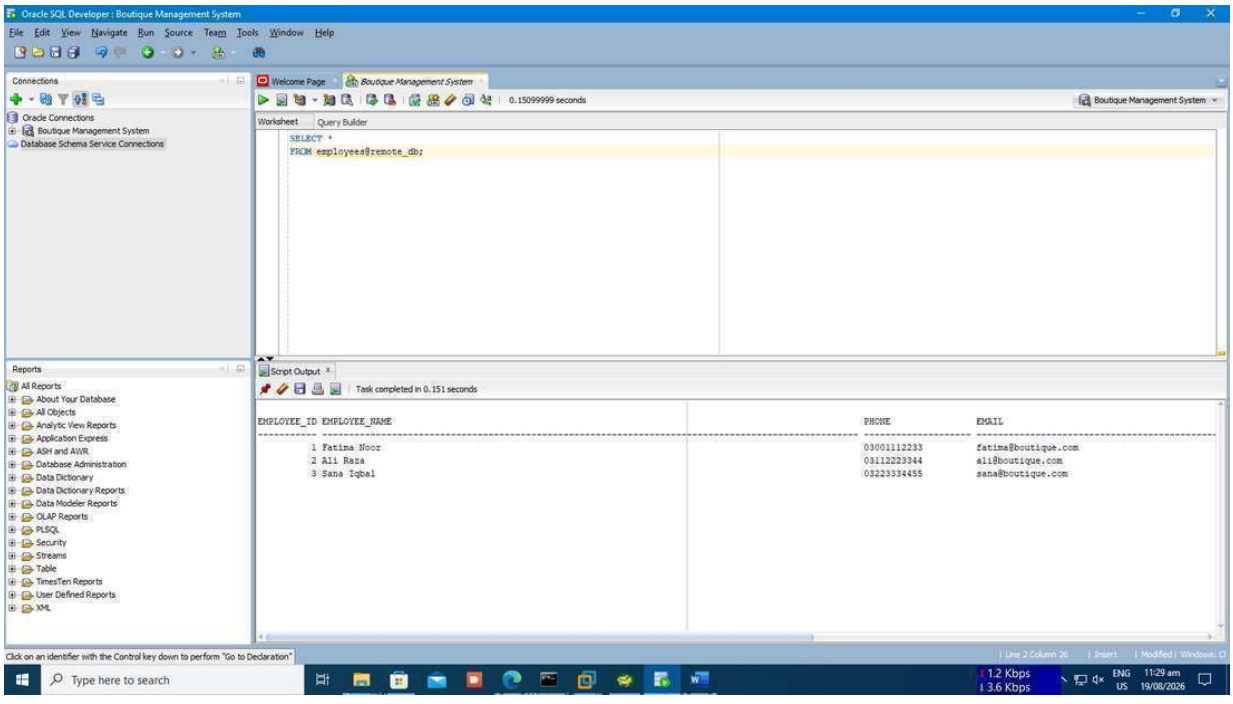
After successfully creating the Database Link, a remote table can be accessed using:

```
SELECT *
FROM employees@remote_db;
```

Explanation

The `@remote_db` portion tells Oracle to access the `EMPLOYEES` table through the specified Database Link.

Screenshot 7.3: Accessing Remote Table



7.1.8 Verifying Database Link

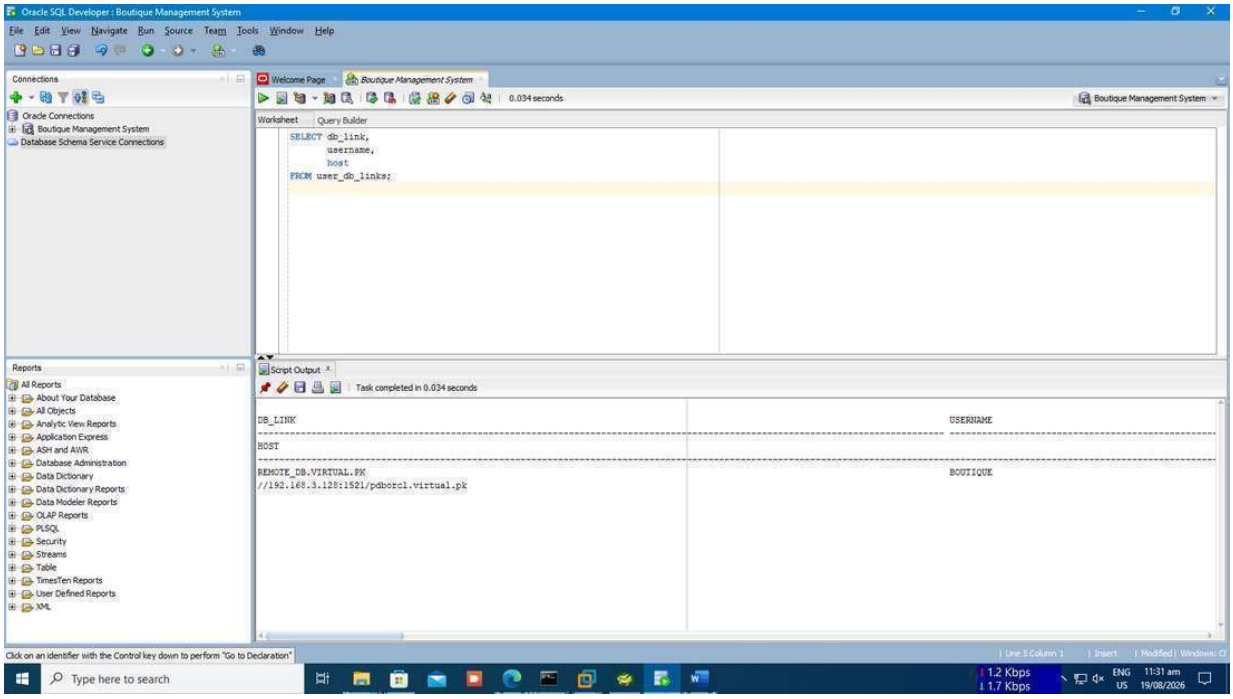
Run:

```
SELECT db_link,
       username,
       host
FROM user_db_links;
```

Explanation

This command displays Database Links owned by the current user.

Screenshot 7.4: Database Link Verification



7.1.9 Troubleshooting Common Database Link Errors

Some common errors include:

ORA-12154

The specified TNS name could not be resolved.

ORA-12514

The listener does not currently know the requested service.

ORA-12541

No listener is running at the specified address.

ORA-01017

Invalid username or password.

These errors can generally be investigated by checking:

- `tnsping`
- Listener status
- Database service
- `tnsnames.ora`
- Username/password
- Network connectivity

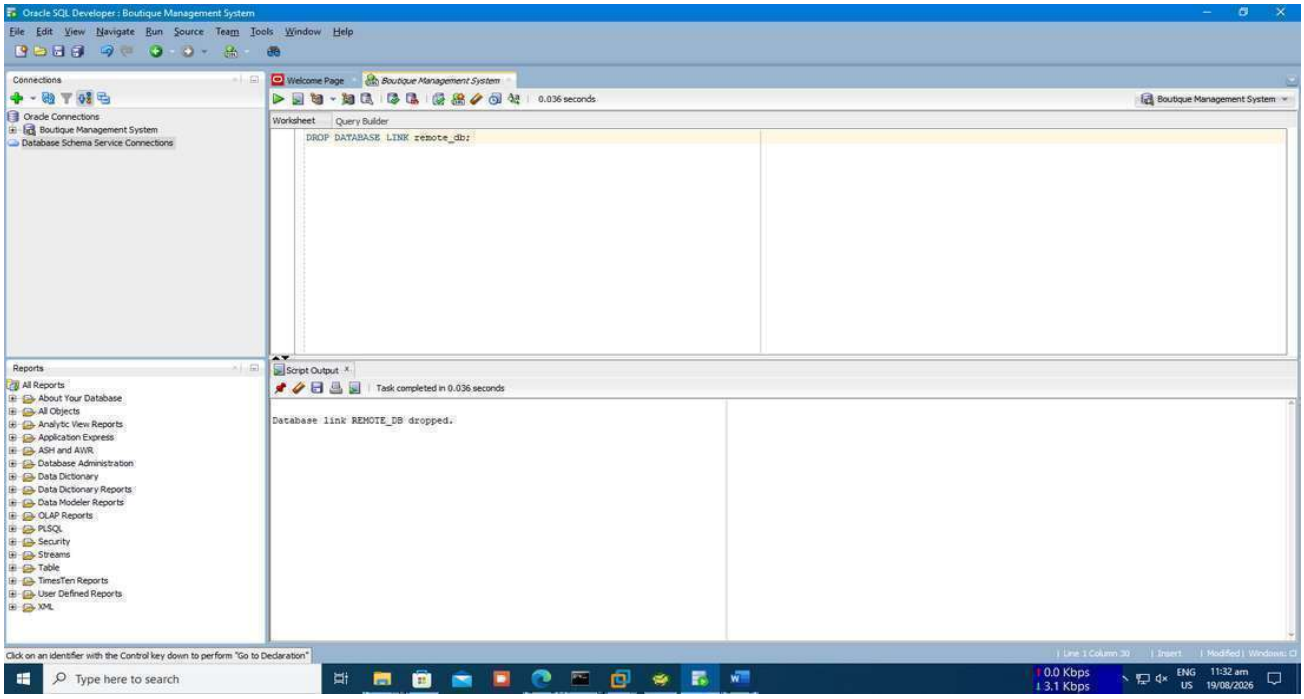
7.1.10 Dropping a Database Link

If the Database Link is no longer required:

```
DROP DATABASE LINK remote_db;
```

Explanation

This command removes the Database Link from the current schema.



7.2 Database Backup and Recovery

Objective

The objective of this practical is to understand Oracle database backup and recovery techniques and to implement different backup methods.

Database backup is one of the most important responsibilities of a Database Administrator because it protects data against accidental deletion, hardware failure, database corruption, and other failures.

7.2.1 Introduction to Database Backup

A database backup is a copy of database information that can be used to restore the database if the original data becomes unavailable or damaged.

A proper backup strategy helps an organization recover its database after a failure.

7.2.2 Importance of Database Backup

Database backups are important because they provide protection against:

- Hardware failure
- Accidental data deletion
- Database corruption
- Operating system failure
- Storage failure
- Human errors
- Disaster situations

A DBA should regularly perform backups and verify that backups can actually be restored.

7.2.3 Types of Database Backup

Oracle database backups can broadly be divided into:

1. **Physical Backup**
2. **Logical Backup**

Other commonly used classifications include:

3. **Cold Backup**
4. **Hot Backup**
5. **Full Backup**
6. **Incremental Backup**
7. **RMAN Backup**
8. **Data Pump Backup**

7.2.4 Comparison of Backup Types

Backup Type	Database Running?	Main Purpose
Physical Backup	Depends on method	Copies physical database files
Logical Backup	Yes	Exports logical objects/data
Cold Backup	No	Offline physical backup
Hot Backup	Yes	Backup while database is available
Full Backup	Usually Yes with RMAN	Complete backup
Incremental Backup	Yes	Backs up changed blocks
RMAN Backup	Yes/depending on operation	Oracle-aware backup and recovery
Data Pump	Yes	Logical export/import

7.2.5 Physical Backup

A physical backup contains physical database files such as:

- Datafiles
- Control files
- Archived redo logs

Physical backups are commonly used for complete database recovery.

7.2.6 Logical Backup

A logical backup contains logical database objects and their data.

Oracle Data Pump is commonly used for logical backup operations.

Examples:

```
EXPDP → Export
IMPDP → Import
```

Logical backups are particularly useful for:

- Schema migration
- Table migration
- Data transfer
- Object-level recovery

7.2.7 Cold Backup

A cold backup is performed while the database is shut down.

Basic concept:

```
Shutdown Database
        ↓
Copy Database Files
        ↓
Backup Completed
        ↓
Start Database
```

Advantage

It is simple and provides a consistent copy of the database files.

Disadvantage

The database must be unavailable during the backup.

7.2.8 Hot Backup

A hot backup is performed while the database remains available to users.

This type of backup is useful for systems that cannot afford significant downtime.

Oracle RMAN is commonly used to perform online backup operations.

7.2.9 Full Backup

7.2.10 Incremental Backup

An incremental backup backs up blocks that have changed since a previous backup.

Level 0 Backup

```
BACKUP INCREMENTAL LEVEL 0 DATABASE;
```

Level 1 Backup

```
BACKUP INCREMENTAL LEVEL 1 DATABASE;
```

Explanation

A Level 0 backup can serve as the baseline for subsequent incremental backups.

A Level 1 backup contains changed blocks since the relevant previous backup.

7.2 RMAN Database Backup

After connecting to the Oracle database using RMAN, a database backup can be performed. Since the database is currently operating in NOARCHIVELOG mode, the database must be shut down consistently and started in MOUNT mode before performing the backup.

7.2.1 Connecting to the Database Using RMAN

First, connect to the target Oracle database using RMAN.

rman target /

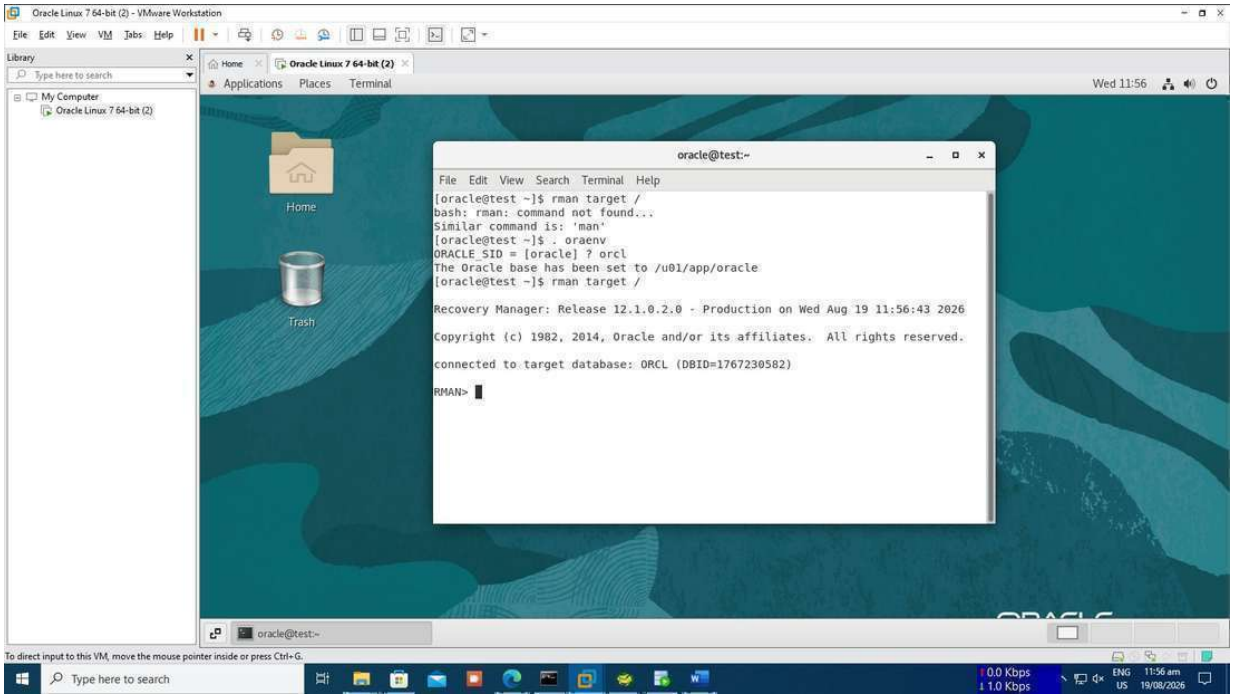
After successful connection, RMAN displays the target database information.

Example:

Recovery Manager: Release 12.1.0.2.0 - Production

connected to target database: ORCL

Figure 7.5: Connecting to the Oracle Database Using RMAN



7.2.2 Attempting Database Backup

Run the following command:

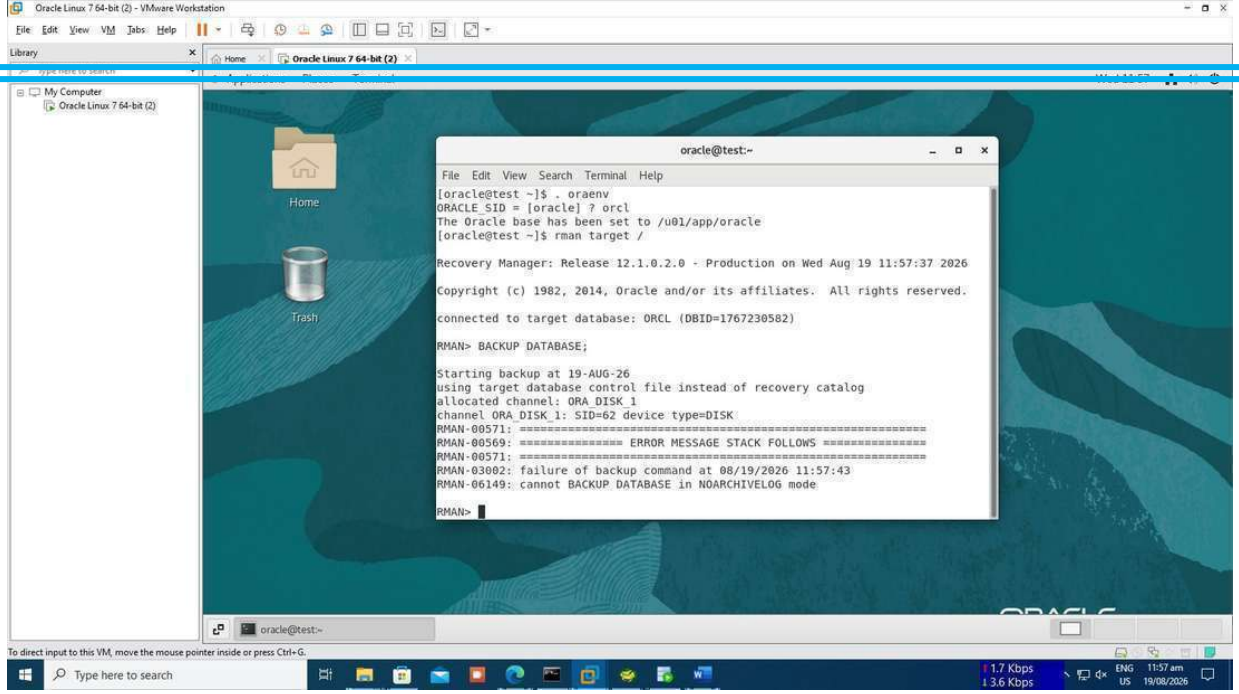
BACKUP DATABASE;

If the database is operating in NOARCHIVELOG mode while it is open, RMAN may return the following error:

RMAN-06149: cannot BACKUP DATABASE in NOARCHIVELOG mode

This occurs because an open database in NOARCHIVELOG mode cannot be backed up using this command. A consistent backup must be performed while the database is mounted.

Figure 7.6: RMAN Backup Error in NOARCHIVELOG Mode



7.2.3 Shutting Down the Database

To perform a consistent backup, shut down the database using:

SHUTDOWN IMMEDIATE;

The SHUTDOWN IMMEDIATE command safely stops the database and disconnects active sessions.

7.2.4 Starting the Database in MOUNT Mode

After shutting down the database, start it in MOUNT mode:

STARTUP MOUNT;

In this state, the database control files are mounted, but the database is not open for normal user access.

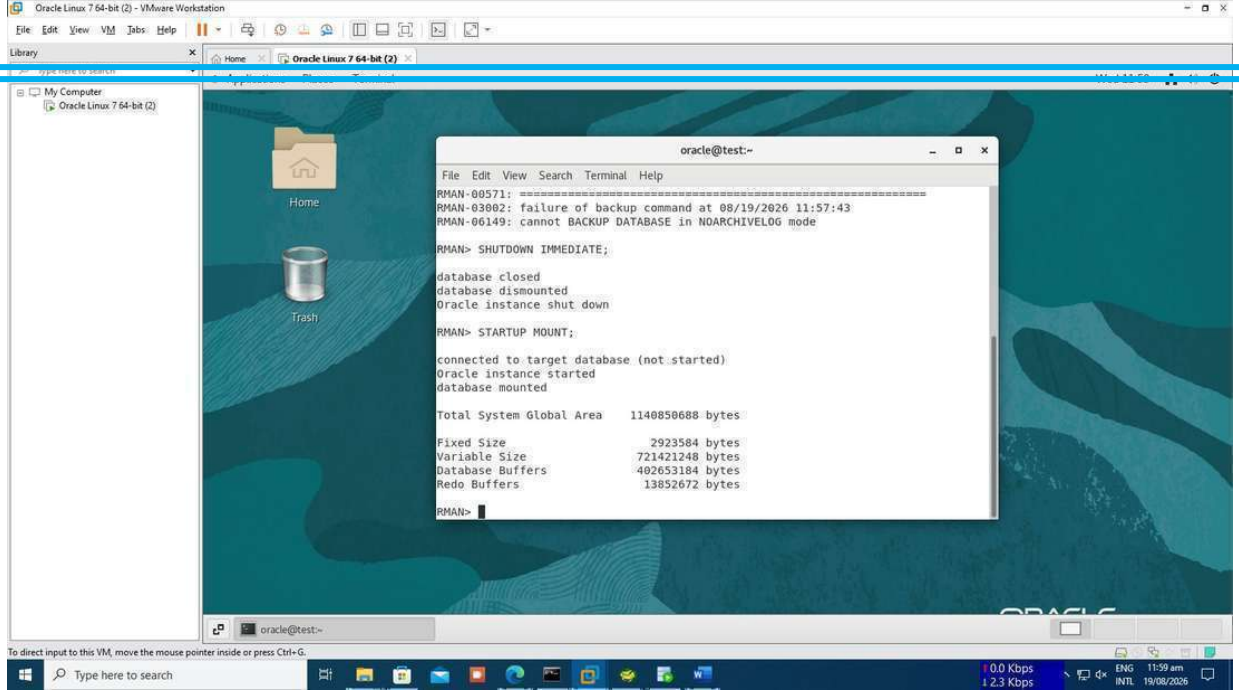
To verify the database status:

SQL 'SELECT STATUS FROM V$INSTANCE';

The expected status is:

MOUNTED

Figure 7.7: Starting the Database in MOUNT Mode



7.2.5 Performing the RMAN Database Backup

Once the database is mounted, run:

BACKUP DATABASE;

RMAN will allocate a disk channel and start backing up the database.

Example:

Starting backup at ...

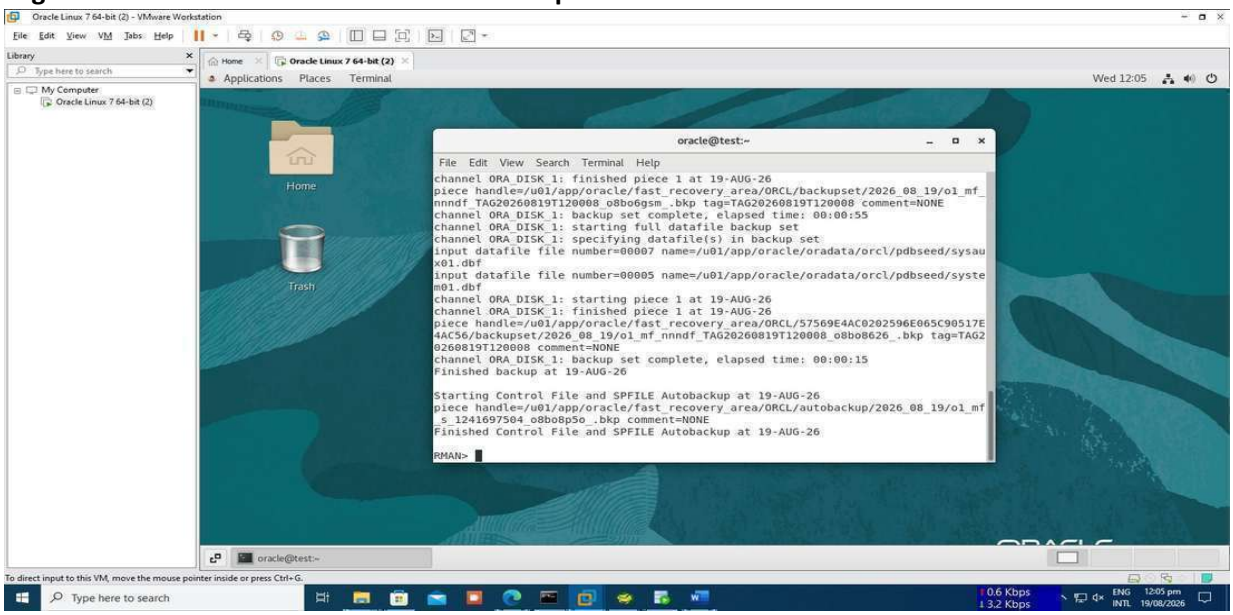
using target database control file instead of recovery catalog

allocated channel: ORA_DISK_1

channel ORA_DISK_1: device type=DISK

After successful completion, RMAN displays information about the backup pieces that were created.

Figure 7.8: Successful RMAN Database Backup



7.2.6 Opening the Database After Backup

After the backup has completed successfully, open the database:

SQL 'ALTER DATABASE OPEN';

The database can now be used normally by users and applications.

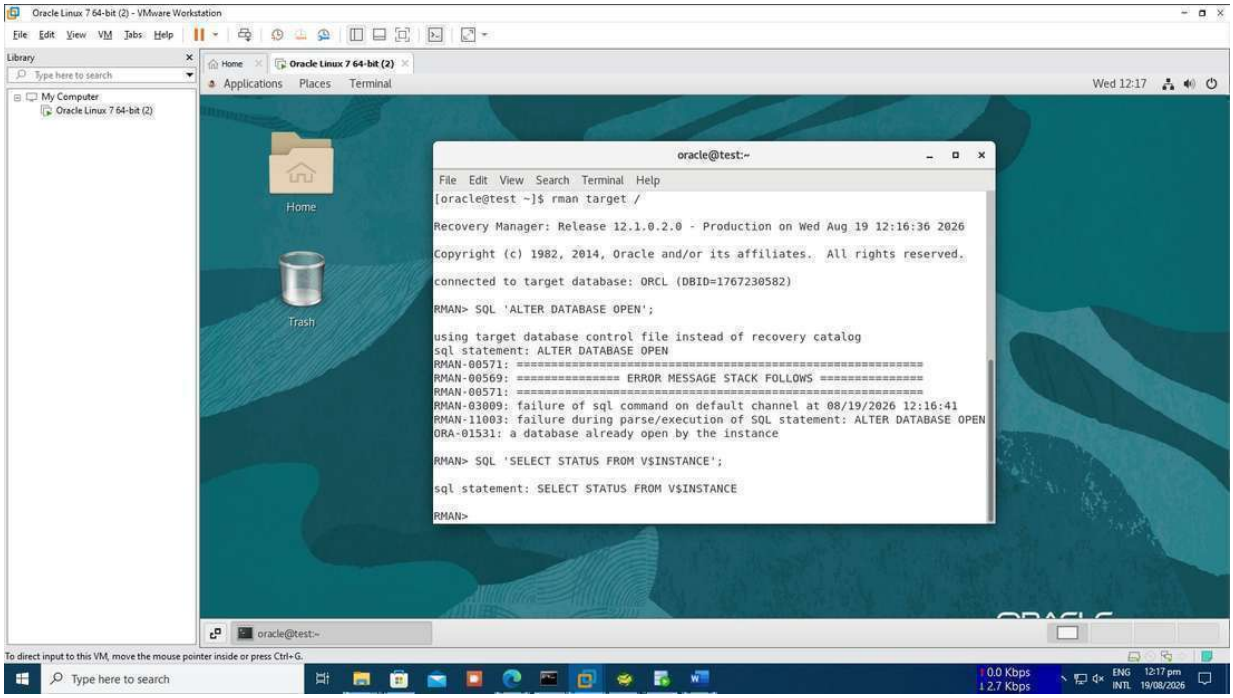
To verify the database status:

SQL 'SELECT STATUS FROM V$INSTANCE';

The expected result is:

OPEN

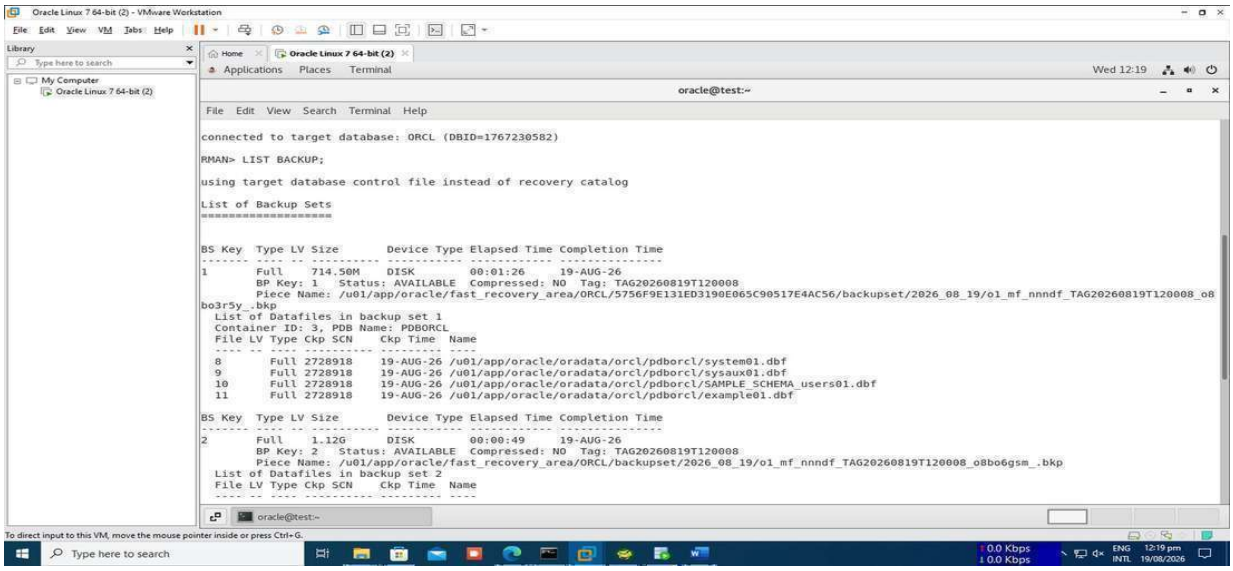
Figure 7.9: Opening and Verifying the Database



Verification

```
LIST BACKUP;
```

Screenshot 7.6: RMAN Backup Verification



7.2.11 RMAN Introduction

RMAN (Recovery Manager) is Oracle's dedicated backup and recovery utility.

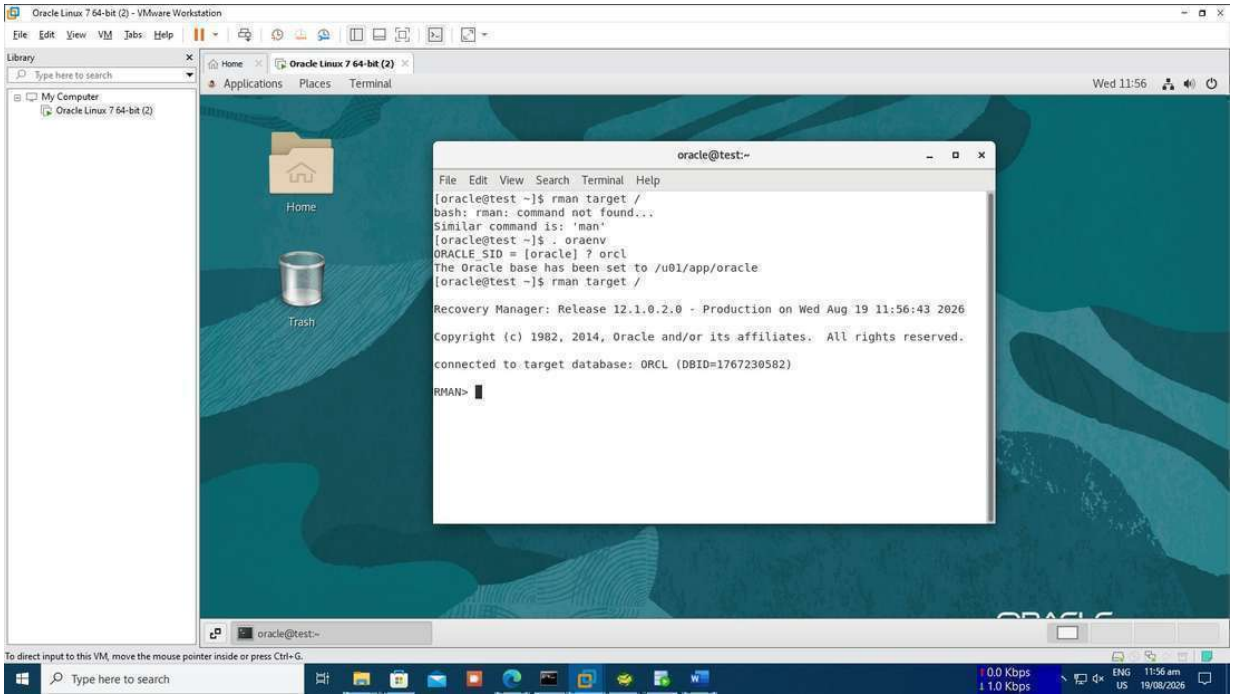
It provides facilities for:

- Database backup
- Incremental backup
- Restore
- Recovery
- Backup verification
- Backup management
- Archived redo log backup

To start RMAN:

```
rman target /
```

Screenshot 7.9: RMAN Connected to Target Database



7.2.12 RMAN Full Database Backup

Connect to RMAN:

```
rman target /
```

Execute:

```
BACKUP DATABASE;
```

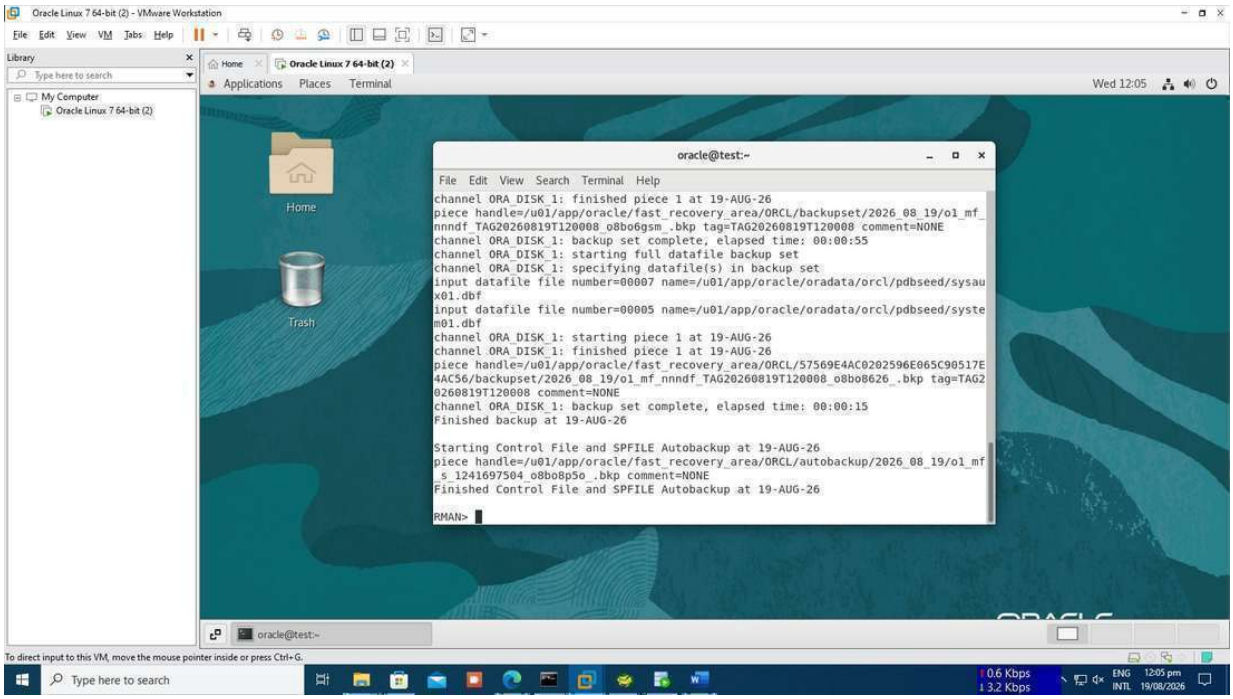
Then verify:

```
LIST BACKUP;
```

Explanation

The `BACKUP DATABASE` command instructs RMAN to create a backup of the database.

Figure 7.8: Successful RMAN Database Backup (After Open Arhievelog Mode)



7.2.13 RMAN Incremental Backup

Create Level 0:

```
BACKUP INCREMENTAL LEVEL 0 DATABASE;
```

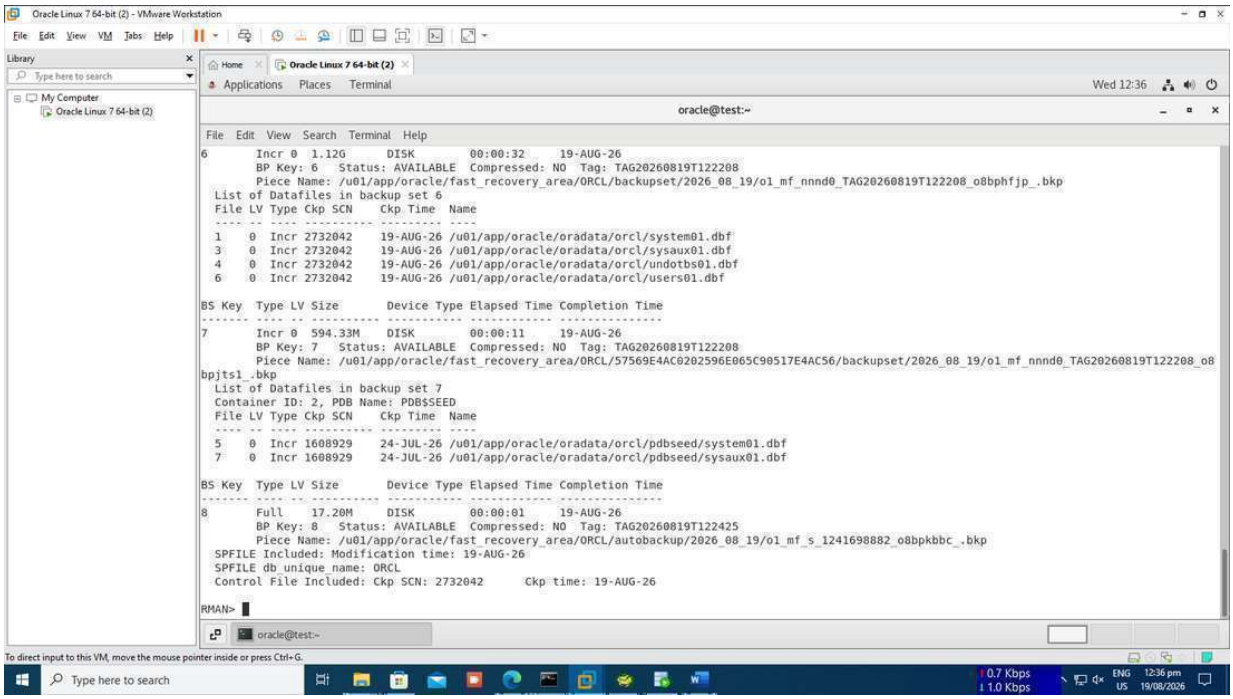
Create Level 1:

```
BACKUP INCREMENTAL LEVEL 1 DATABASE;
```

Verify:

```
LIST BACKUP;
```

Screenshot 7.10: Incremental Backup



7.2.14 Control File Backup

The control file contains important structural information about the database.

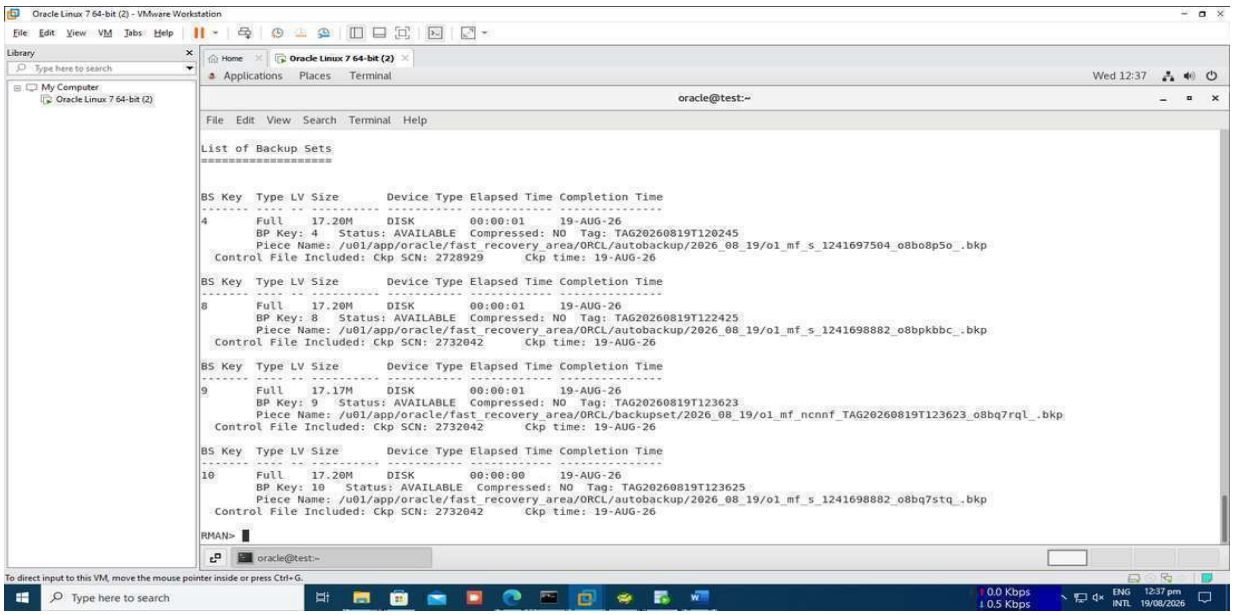
Backup command:

```
BACKUP CURRENT CONTROLFILE;
```

Verify:

```
LIST BACKUP OF CONTROLFILE;
```

Screenshot 7.11: Control File Backup



7.2.15 Archive Log Backup

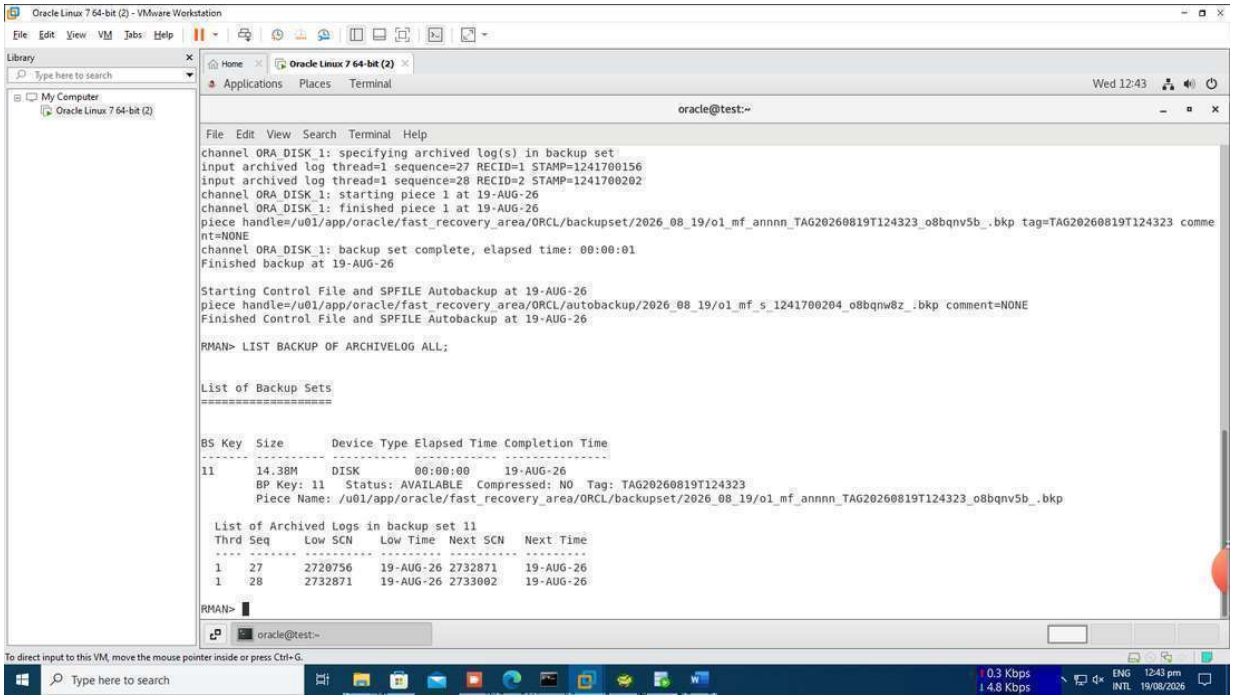
If the database is configured in ARCHIVELOG mode:

```
BACKUP ARCHIVELOG ALL;
```

Verify:

```
LIST BACKUP OF ARCHIVELOG ALL;
```

Screenshot 7.12: Archive Log Backup



7.2.16 Backup Verification

RMAN provides several commands for checking backups.

```
LIST BACKUP;
```

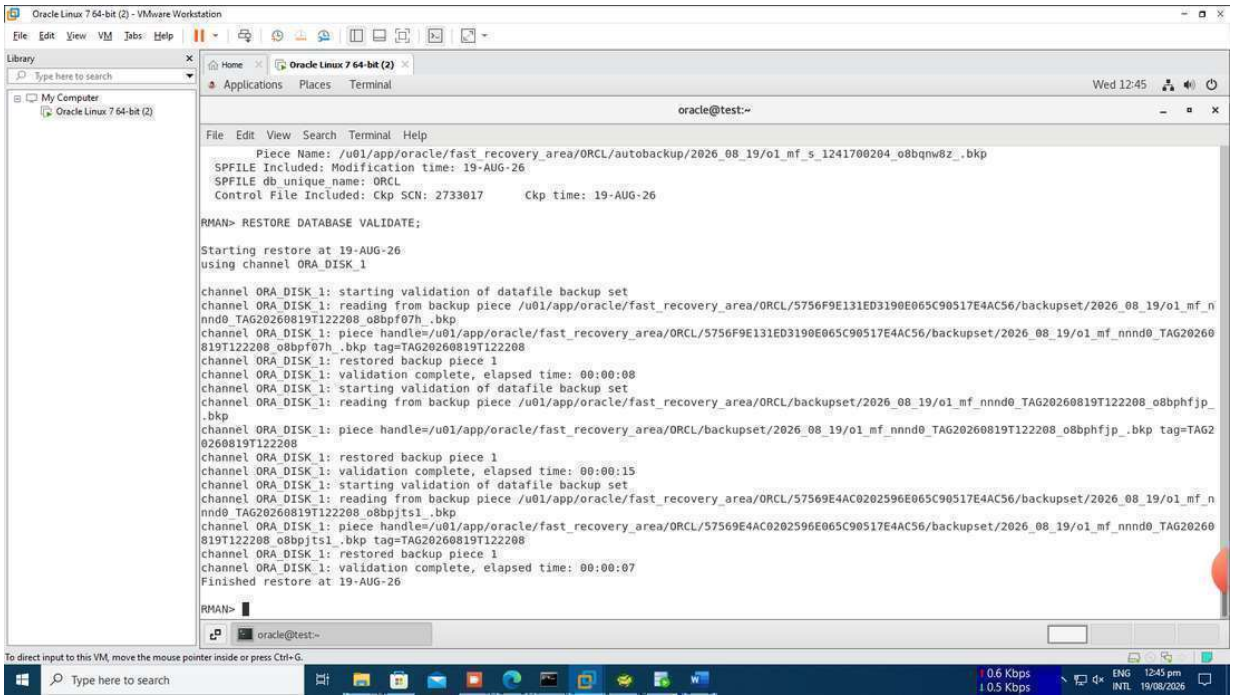
For backup validation:

```
RESTORE DATABASE VALIDATE;
```

Explanation

Backup validation helps determine whether RMAN can use the required backup files for restoration.

Screenshot 7.13: Backup Validation



7.2.17 Restore and Recovery

Backup is only one part of a backup strategy. A DBA must also understand how to restore and recover a database.

The general process is:

```
Backup
   ↓
Database Failure
   ↓
Restore
   ↓
Recovery
   ↓
Database Available
```

Restore Example

A typical RMAN restore command is:

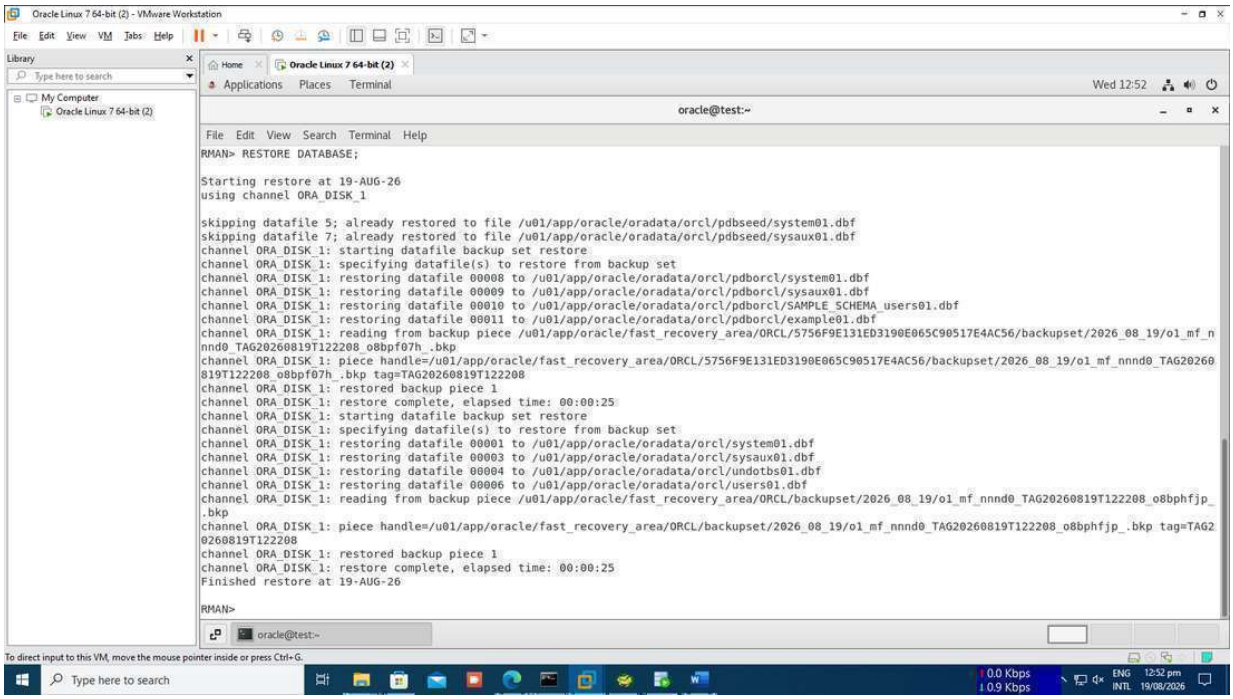
```
RESTORE DATABASE;
```

Recovery:

```
RECOVER DATABASE;
```

Important: Actual destructive recovery testing should only be performed in a dedicated lab environment. We will not intentionally damage the main Boutique database just to demonstrate recovery.

Screenshot 7.14: Restore/Recovery Demonstration



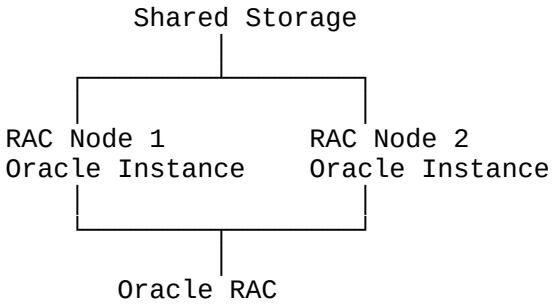
7.2.18 RAC Introduction

Oracle Real Application Clusters (RAC) allows multiple Oracle instances on different nodes to work with a shared database.

RAC is mainly used to provide:

- High availability
- Failover
- Scalability
- Load balancing

7.2.19 RAC Architeture

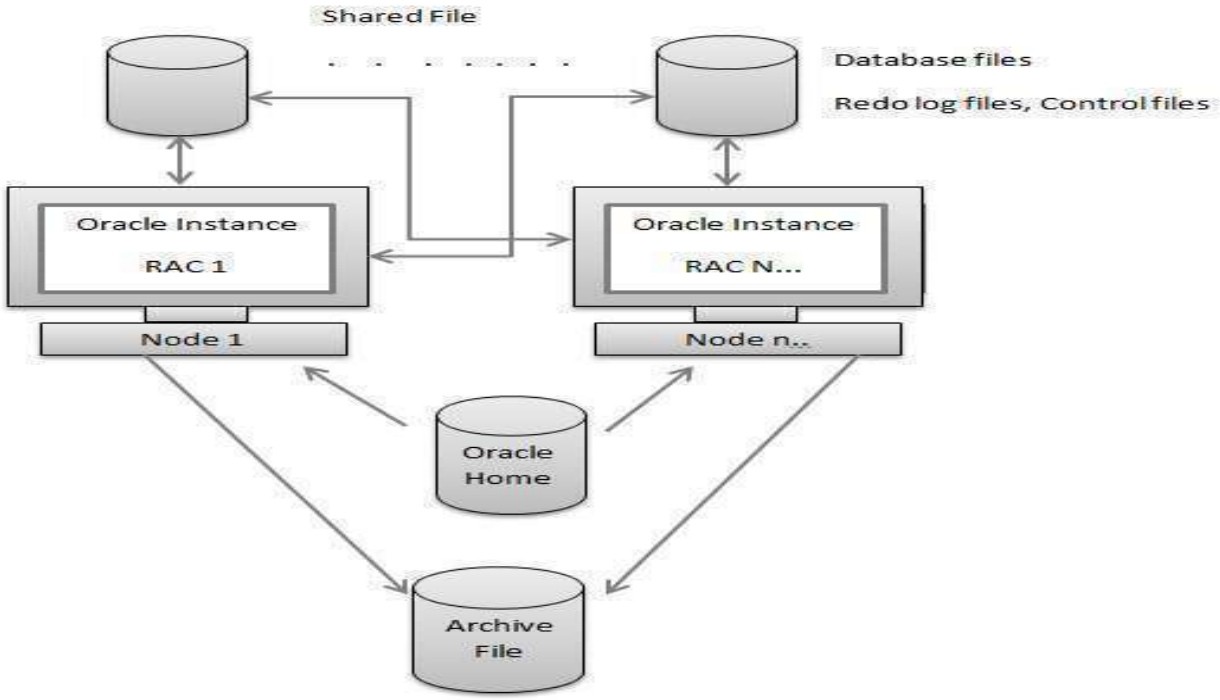


Explanation

Multiple Oracle instances operate against the same database.

If one node becomes unavailable, the remaining node(s) can continue serving the database workload, subject to the configured architecture.

Screenshot/Figure 7.15: Oracle RAC Architecture



Important: RAC will be discussed conceptually in this practical. We will not install RAC on the current single-VM lab environment.

7.2.20 RAC vs Single Instance

Feature	Single Instance	Oracle RAC
Number of instances	One	Multiple
Nodes	Usually one	Multiple
Availability	Normal	High
Failover capability	Limited	High availability design
Configuration	Simple	Complex
Infrastructure	Lower	Higher
Administration	Easier	More advanced

Practical No. 10: Database Monitoring and Performance Tuning

7.3 Database Monitoring and Performance Tuning

Objective

The objective of this practical is to monitor the Oracle database and understand basic techniques for identifying and improving database performance.

7.3.1 Introduction to Database Monitoring

Database monitoring is the process of observing the health, availability, resource usage, sessions, storage, and performance of a database.

A DBA uses monitoring to identify problems before they affect users.

7.3.2 Monitoring Database Sessions

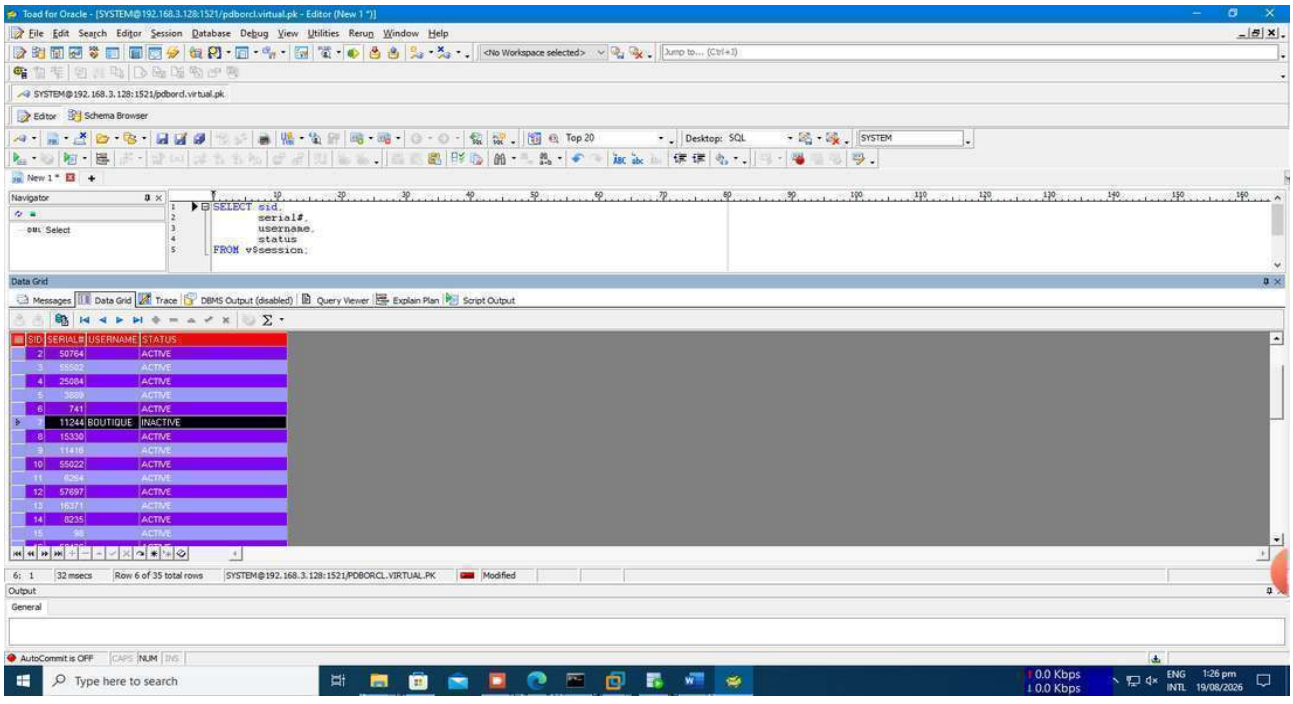
Run:

```
SELECT sid,
       serial#,
       username,
       status
FROM v$session;
```

Explanation

This query displays active database sessions and their current status.

📷 Screenshot 7.16: Database Session Monitoring



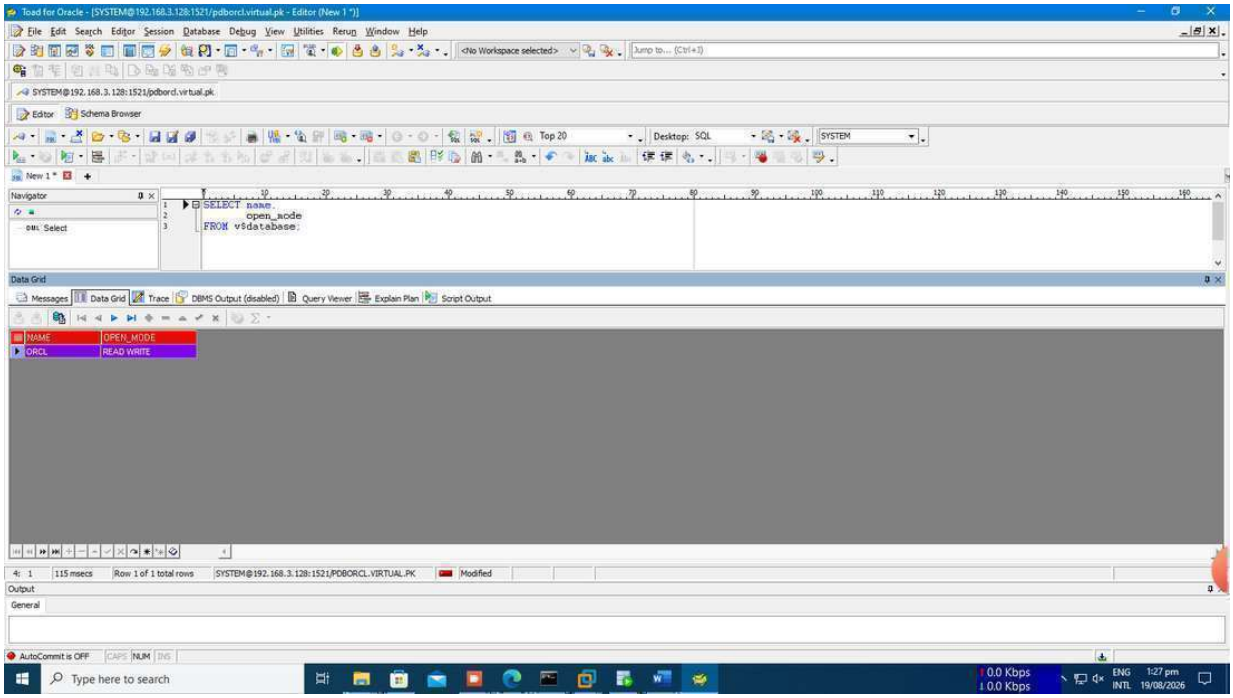
7.3.3 Checking Database Status

```
SELECT name,
       open_mode
FROM v$database;
```

Explanation

This command displays the database name and its current open mode.

Screenshot 7.17: Database Status



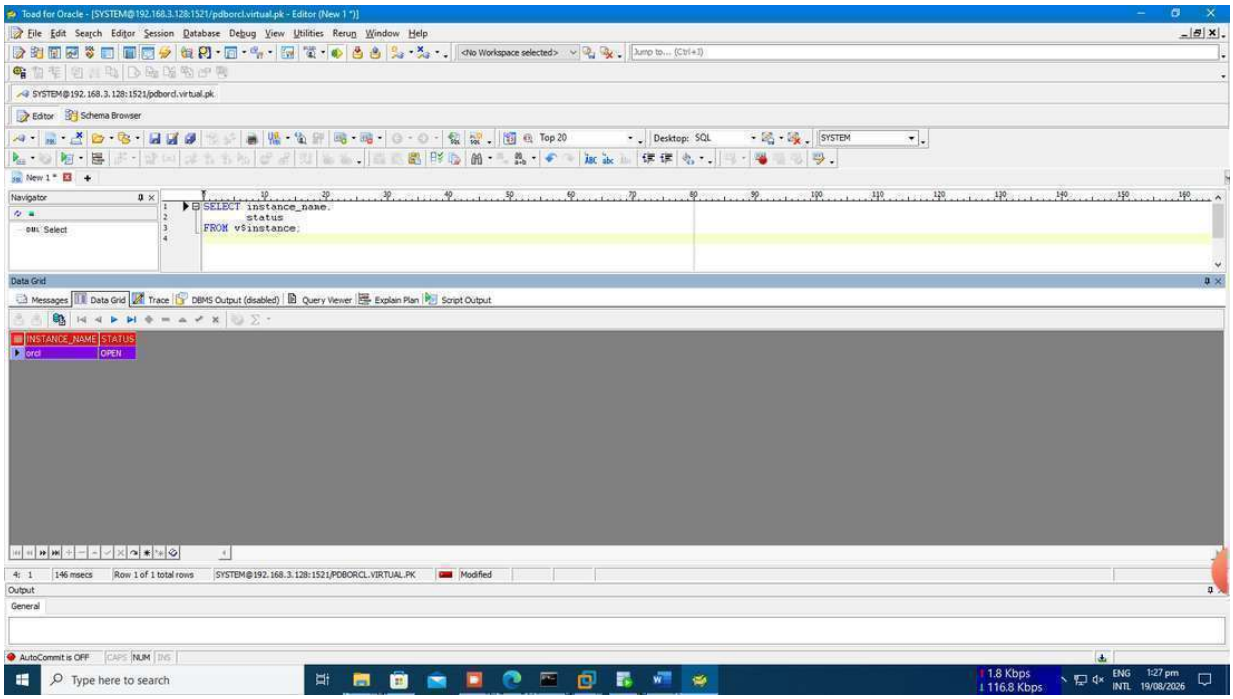
7.3.4 Checking Instance Status

```
SELECT instance_name,
       status
FROM v$instance;
```

Explanation

This query displays the Oracle instance name and its current status.

Screenshot 7.18: Instance Status



7.3.5 Tablespace Monitoring

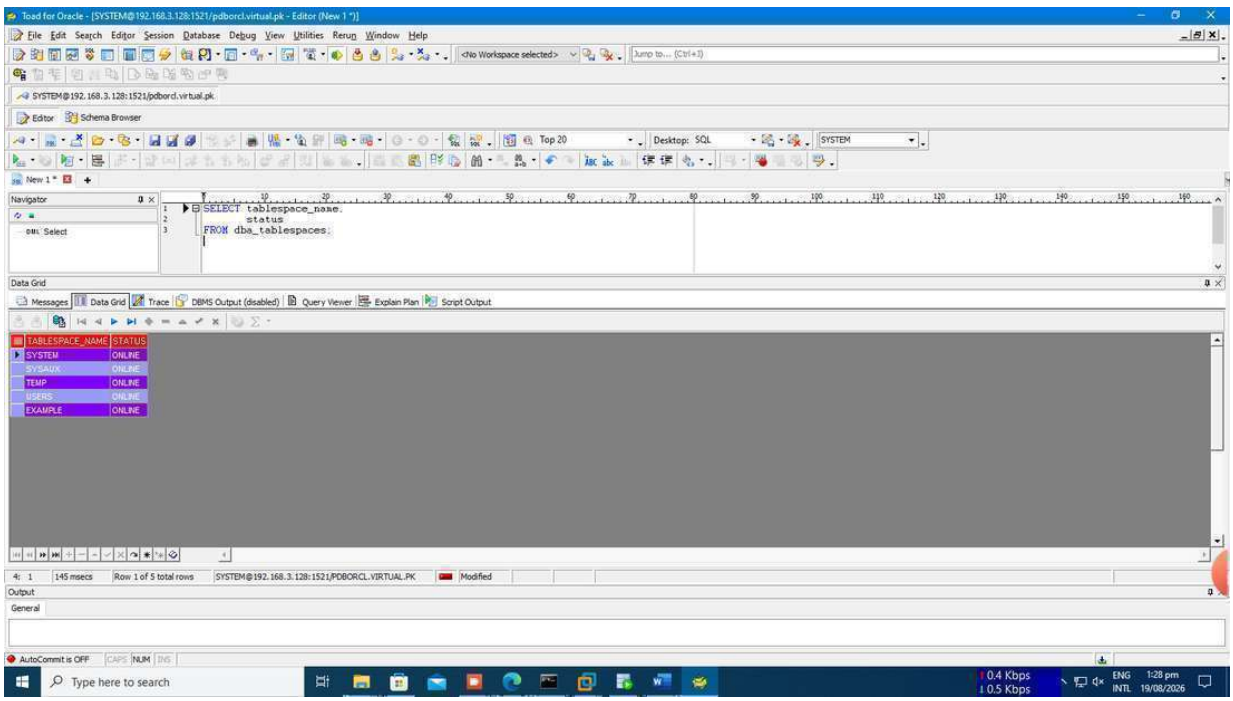
```
SELECT tablespace_name,
       status
FROM dba_tablespaces;
```

Explanation

This query displays the available tablespaces and their status.

Tablespace monitoring is important because insufficient storage can affect database operations.

📷 Screenshot 7.19: Tablespace Monitoring



7.3.6 SQL Performance Monitoring

A DBA should identify SQL statements that consume excessive resources.

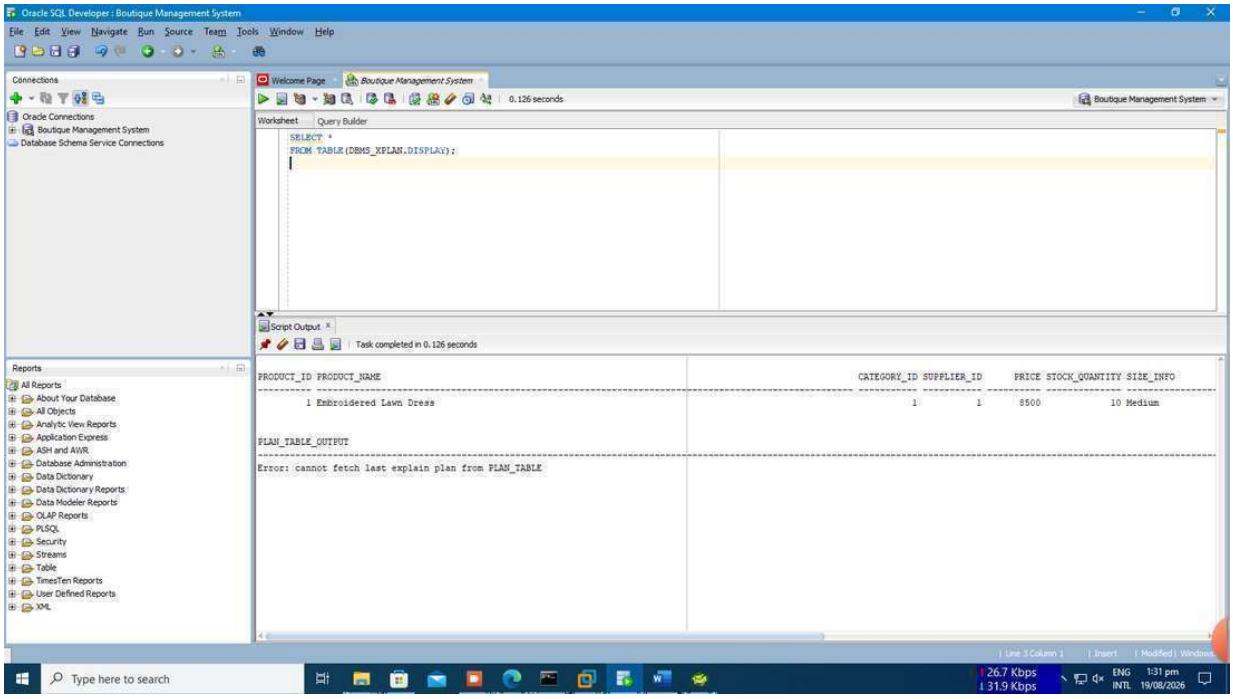
A basic example is to inspect an execution plan.

```
EXPLAIN PLAN FOR
SELECT *
FROM products
WHERE product_id = 1;
```

Then:

```
SELECT *
FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Screenshot 7.20: SQL Execution Plan



7.3.7 Execution Plan

An execution plan shows how Oracle intends to execute a SQL statement.

It can help a DBA understand:

- Table access
- Index usage
- Join methods
- Estimated rows
- Cost

Execution plans are an important part of SQL performance tuning.

7.3.8 Introduction to Performance Tuning

Performance tuning aims to improve database response time and efficient use of resources.

Important areas include:

- SQL optimization
- Indexes
- Execution plans
- Memory
- CPU usage
- I/O
- Wait event

7.3.9 SQL Optimization

Poorly written SQL can consume unnecessary database resources.

For example, queries should be designed carefully and appropriate filtering conditions should be used.

Example:

```
SELECT product_name,
       price
FROM products
WHERE product_id = 1;
```

The execution plan can then be examined to understand how Oracle accesses the table.

7.3.10 Indexes and Performance

Indexes can improve the performance of queries that frequently search or join on indexed columns.

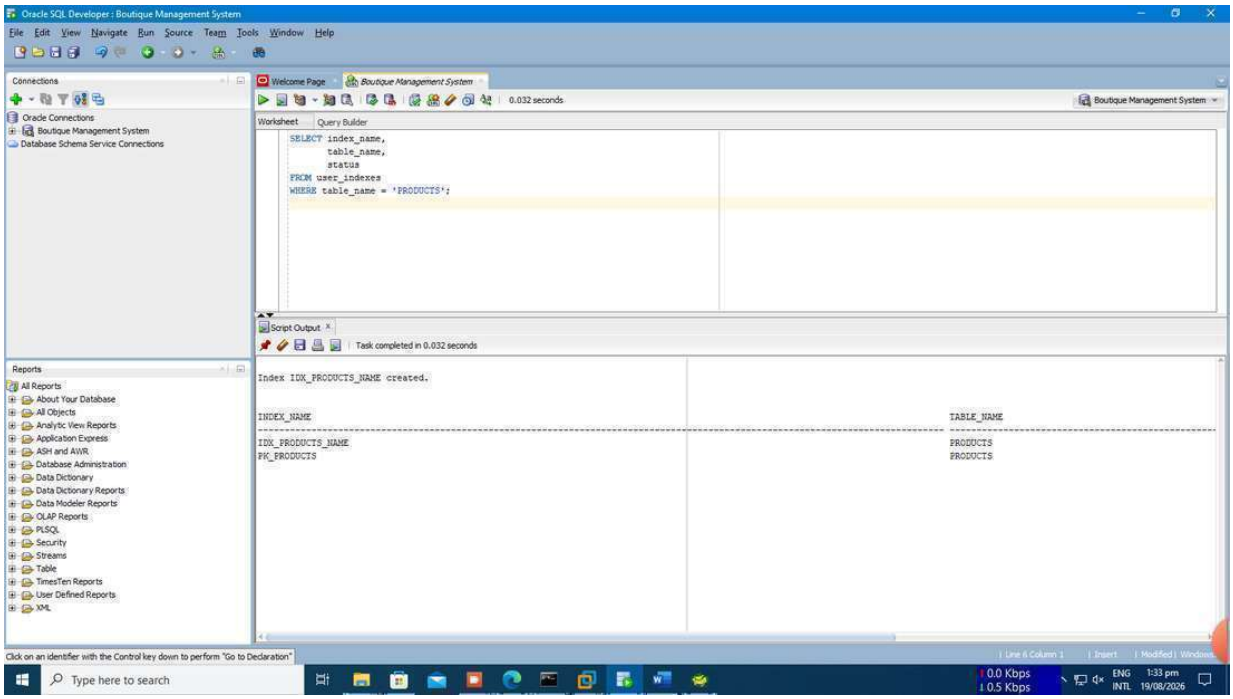
Example:

```
CREATE INDEX idx_products_name
ON products(product_name);
```

Verify:

```
SELECT index_name,
       table_name,
       status
FROM user_indexes
WHERE table_name = 'PRODUCTS';
```

Screenshot 7.21: Index Creation and Verification



Important: Indexes should not be created unnecessarily. They can improve reads but also add storage and maintenance overhead.

7.3.11 AWR, ASH and ADDM Introduction

Oracle provides several performance analysis features:

AWR — Automatic Workload Repository

Stores historical performance information.

ASH — Active Session History

Provides information about active database sessions over time.

ADDM — Automatic Database Diagnostic Monitor

Analyzes database performance data and can identify performance-related problems.

These tools are important concepts for Oracle DBA roles.

Note: Availability and access to these features depend on the Oracle Database edition, licensing, configuration, and privileges. We will only demonstrate features that are available and appropriate in the current Oracle 12c lab environment.

Practical No. 11: Database Security and User/Privilege Management

7.4 Database Security and User Management

Objective

The objective of this practical is to understand Oracle database security and learn how to manage database users, roles, system privileges, and object privileges.

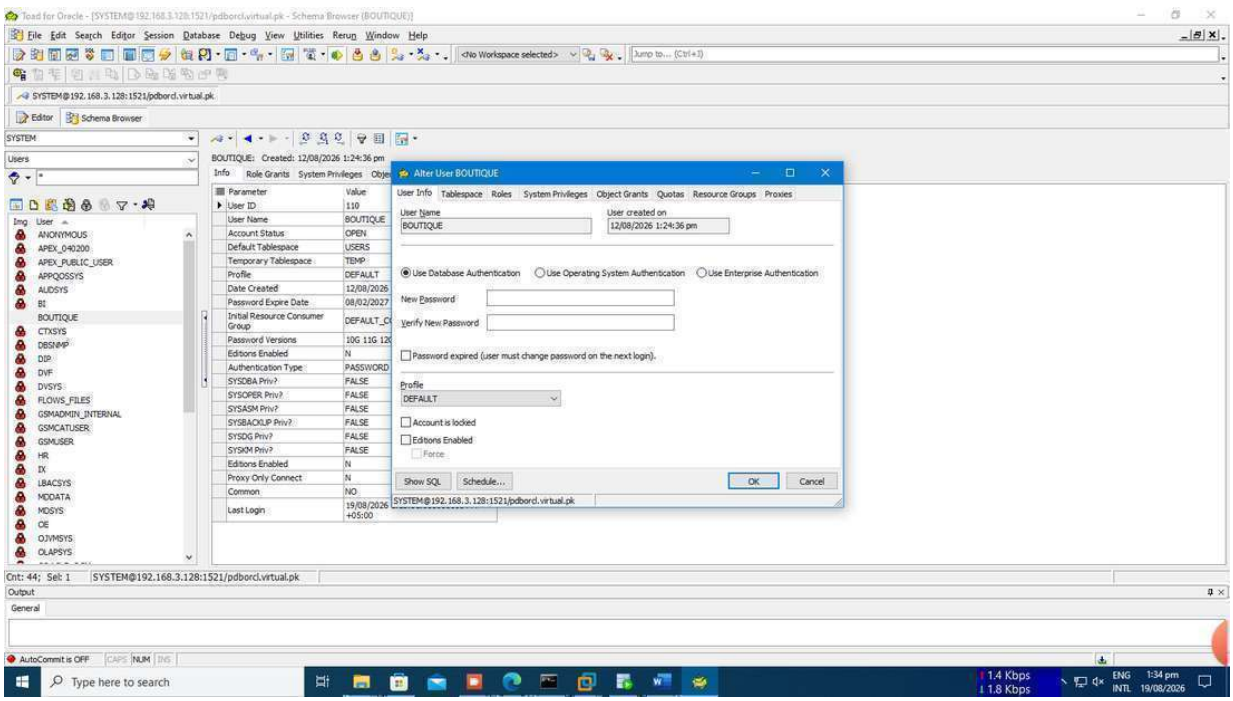
7.4.1 Introduction to Database Security

Database security protects database information from unauthorized access, modification, or deletion.

Oracle provides different security mechanisms, including:

- Users
- Roles
- System privileges
- Object privileges
- Authentication
- Authorization
- Password management

Screenshot 7.23: Database Security and User Management Environment



7.4.2 Database Users

A database user is an account that is used to connect to an Oracle database.

In our **Boutique Management System**, the main project schema is:

`BOUTIQUE`

The user can be viewed through SQL Developer or TOAD.

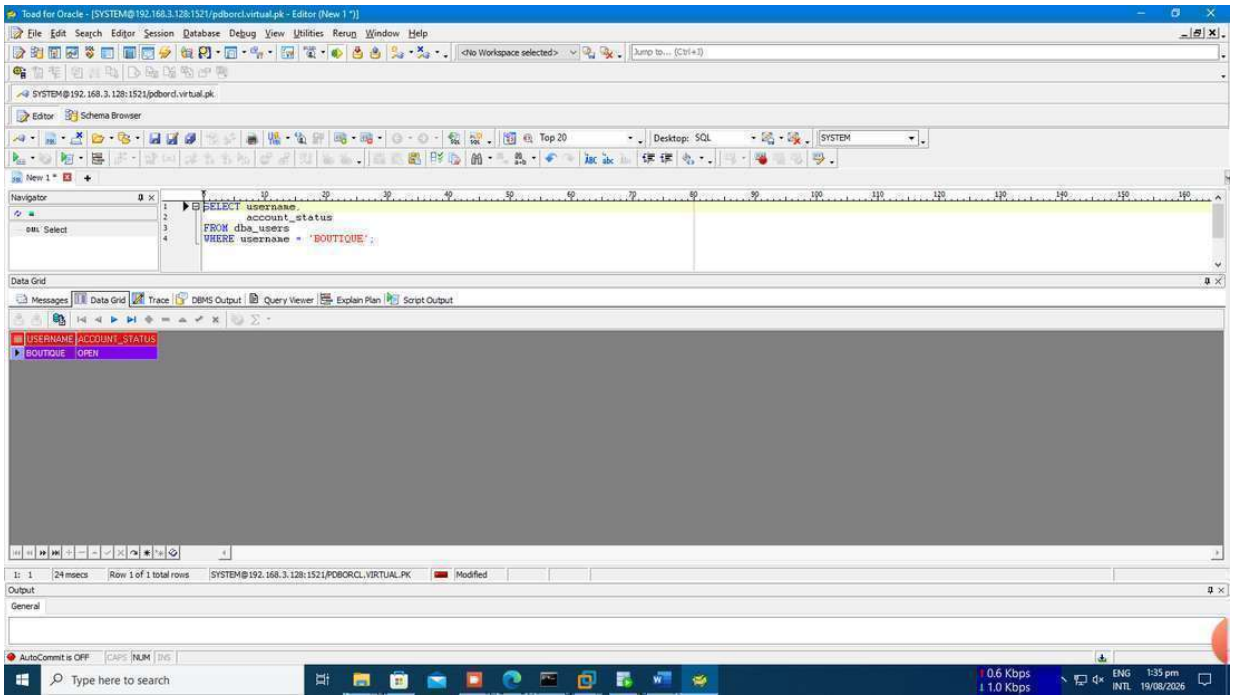
SQL Command

```
SELECT username,
       account_status
FROM dba_users
WHERE username = 'BOUTIQUE';
```

Explanation

This command displays the `BOUTIQUE` user and its account status.

Screenshot 7.24: BOUTIQUE User Verification



7.4.3 Creating a Database User

For demonstration purposes, a test DBA user can be created.

Command

```
CREATE USER test1
IDENTIFIED BY password123;
```

Explanation

The `CREATE USER` command creates a new Oracle database user named `TEST_DBA`.

Screenshot 7.25: Creating TEST1 User

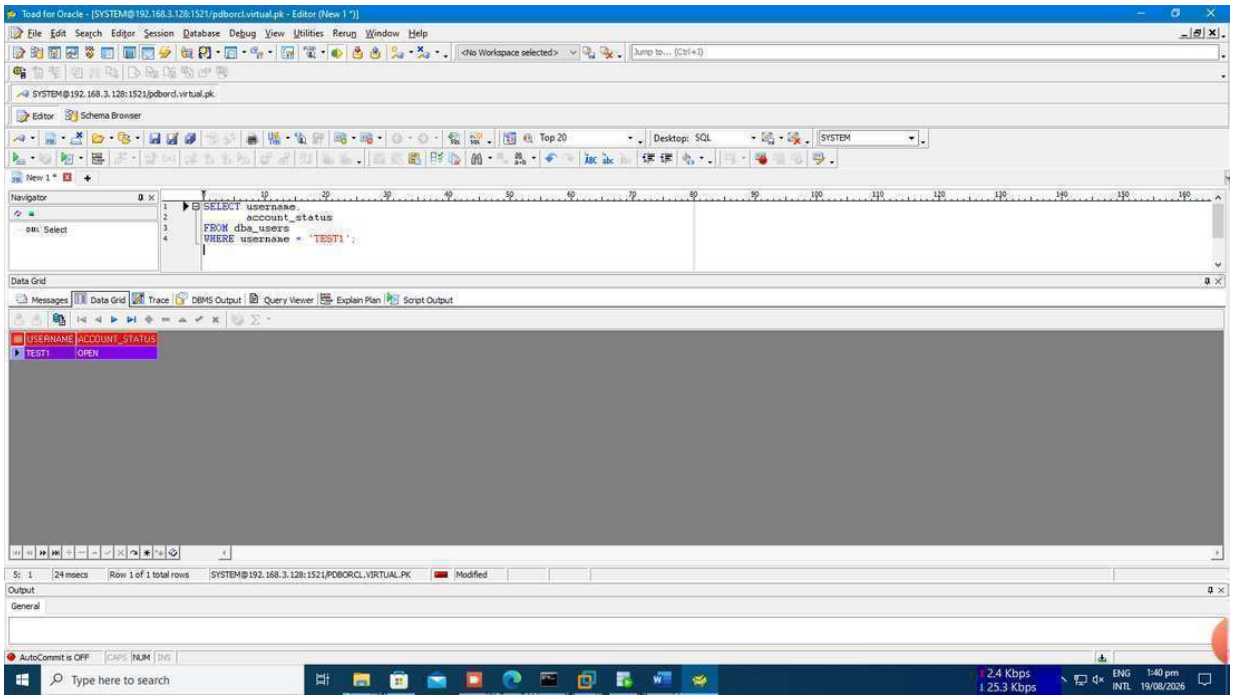
Verification

```
SELECT username,
       account_status
FROM dba_users
WHERE username = 'TEST1';
```

Explanation

This query verifies that the `TEST_DBA` user has been successfully created.

Screenshot 7.26: TEST_DBA User Verification



7.4.4 Database Roles

A role is a collection of privileges that can be assigned to a user.

Roles make privilege management easier because multiple privileges can be grouped together.

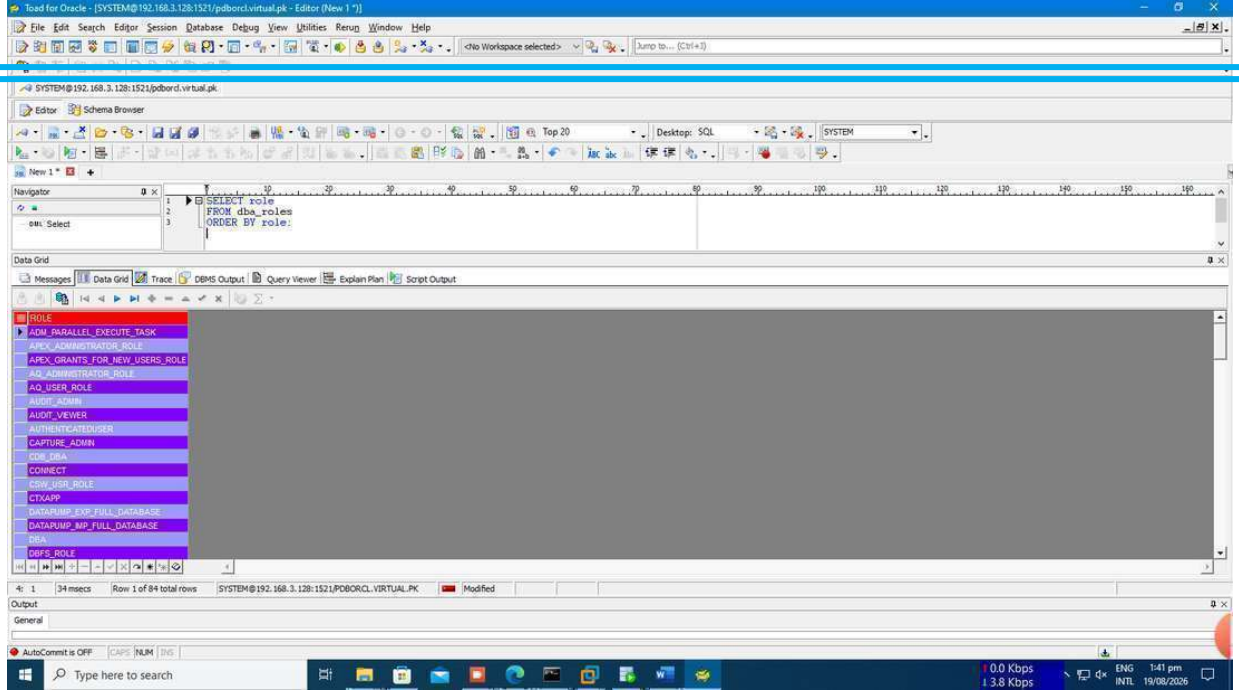
View Available Roles

```
SELECT role
FROM dba_roles
ORDER BY role;
```

Explanation

This command displays the roles available in the Oracle database.

Screenshot 7.27: Database Roles



7.4.5 System Privileges

System privileges allow users to perform specific database-level operations.

Examples include:

```
CREATE SESSION
CREATE TABLE
CREATE VIEW
CREATE SEQUENCE
```

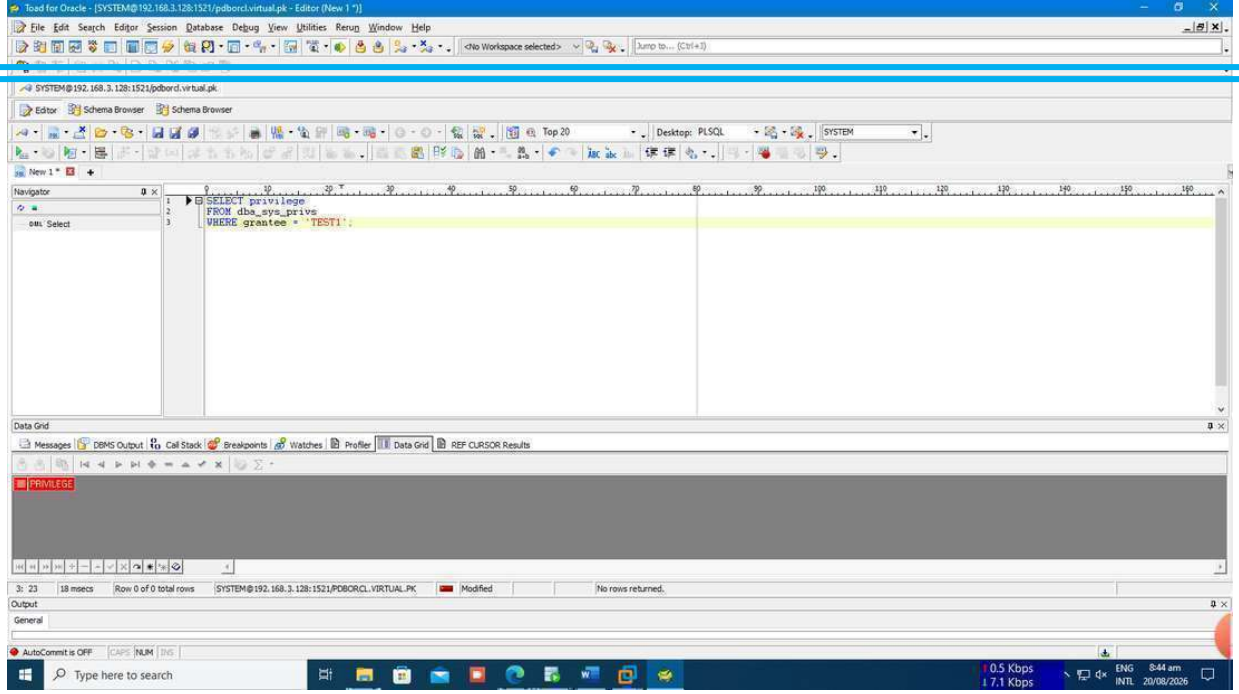
View User Privileges

```
SELECT privilege
FROM dba_sys_privs
WHERE grantee = 'TEST1';
```

Explanation

This query displays the system privileges currently granted to `TEST_DBA`.

Screenshot 7.28: System Privileges of TEST_DBA



7.4.6 Granting CREATE SESSION Privilege

A newly created user requires the appropriate privilege to connect to the database.

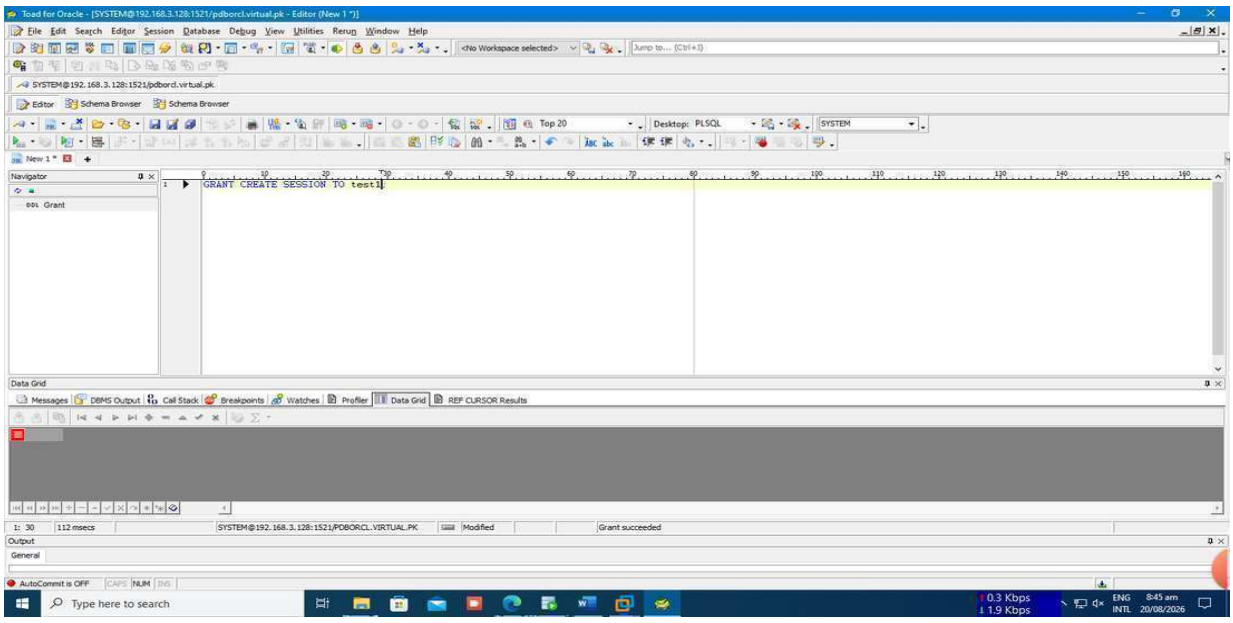
Command

```
GRANT CREATE SESSION
TO test_dba;
```

Explanation

The `CREATE SESSION` privilege allows the user to establish a database session.

Screenshot 7.29: CREATE SESSION Privilege Granted

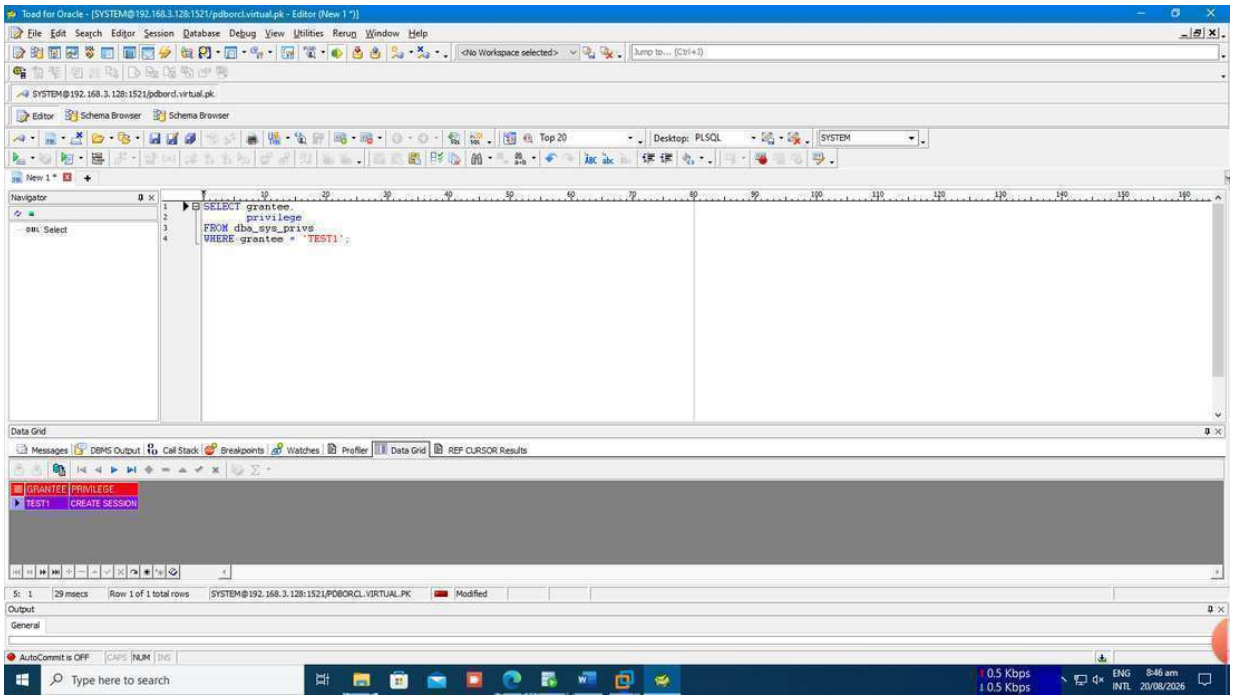


Verification

```
SELECT grantee,
```

```
       privilege
FROM dba_sys_privs
WHERE grantee = 'TEST1';
```

Screenshot 7.30: CREATE SESSION Privilege Verification



7.4.7 Object Privileges

Object privileges provide access to specific database objects such as tables and views.

Common object privileges include:

```
SELECT
INSERT
UPDATE
DELETE
```

For example, we can provide `SELECT` access to the `PRODUCTS` table.

7.4.8 Granting SELECT Privilege

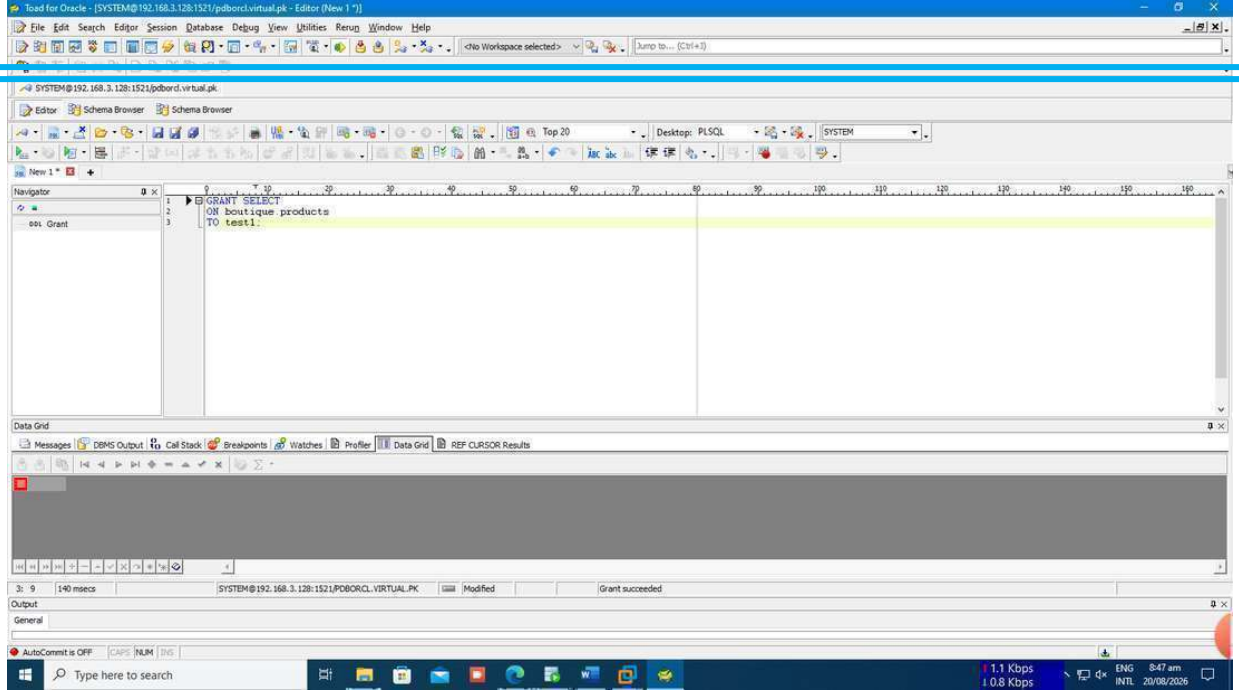
Command

```
GRANT SELECT
ON boutique.products
TO test1;
```

Explanation

This command gives `TEST1` permission to retrieve data from the `BOUTIQUE.PRODUCTS` table.

Screenshot 7.31: SELECT Privilege Granted on PRODUCTS



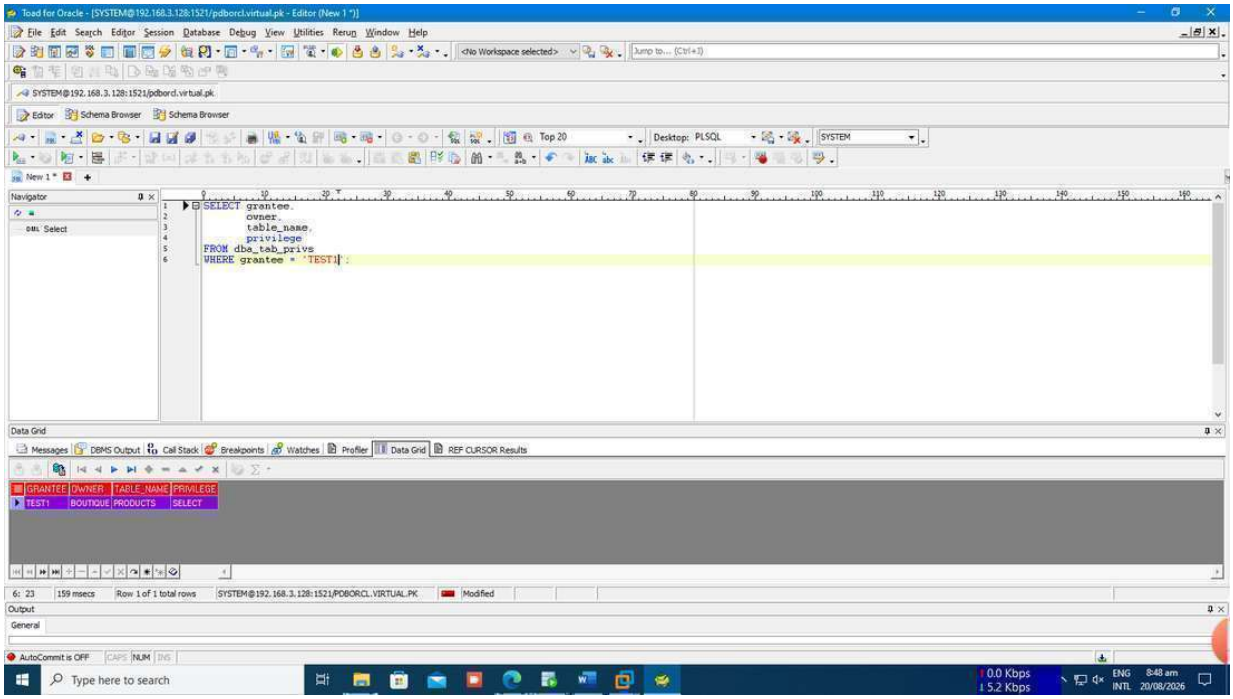
Verification

```
SELECT grantee,
       owner,
       table_name,
       privilege
FROM dba_tab_privs
WHERE grantee = 'TEST1';
```

Explanation

This query verifies the object privileges granted to `TEST1`.

📷 Screenshot 7.32: Object Privilege Verification



7.4.9 Testing the SELECT Privilege

Now connect to the database using the `TEST1` user.

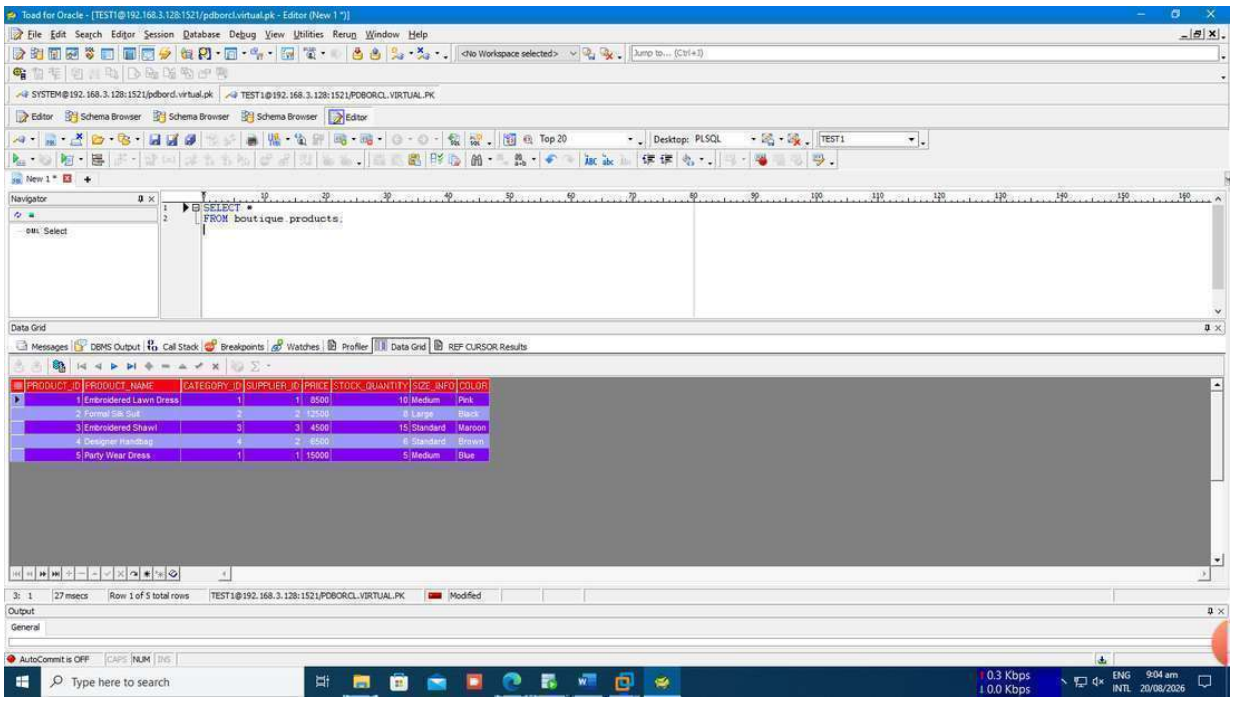
Run:

```
SELECT *
FROM boutique.products;
```

Explanation

If the `SELECT` privilege was granted successfully, the user should be able to retrieve records from the `PRODUCTS` table.

Screenshot 7.33: TEST1 Accessing PRODUCTS Table



7.4.10 Testing Unauthorized Access

To demonstrate database security, try accessing an object for which the user has not been given permission.

For example:

```
SELECT *
FROM boutique.customers;
```

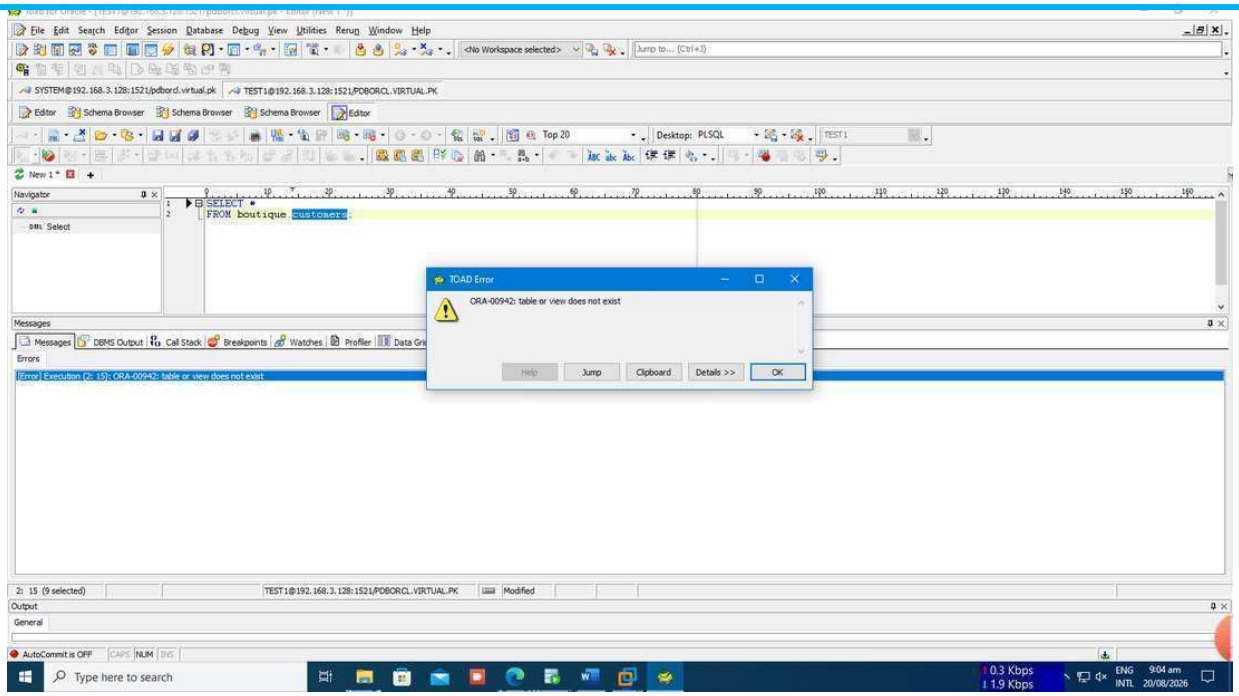
Expected Result

If `TEST_DBA` has not been granted `SELECT` privilege on the `CUSTOMERS` table, Oracle should reject the operation with an insufficient privileges error.

Explanation

This demonstrates that Oracle prevents users from accessing database objects for which they do not have the required privileges.

Screenshot 7.34: Unauthorized Access Error



7.4.11 Revoking Object Privilege

The `REVOKE` command is used to remove a previously granted privilege.

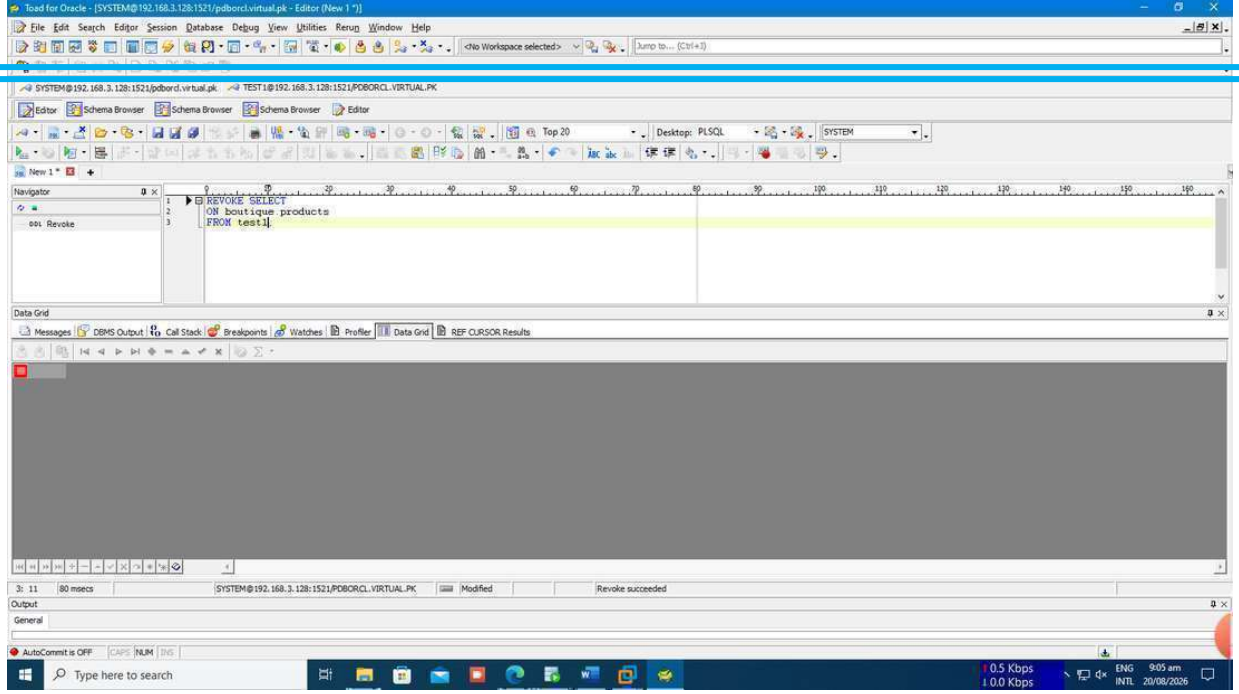
Command

```
REVOKE SELECT
ON boutique.products
FROM test_dba;
```

Explanation

This command removes the `SELECT` privilege from `TEST_DBA` for the `PRODUCTS` table.

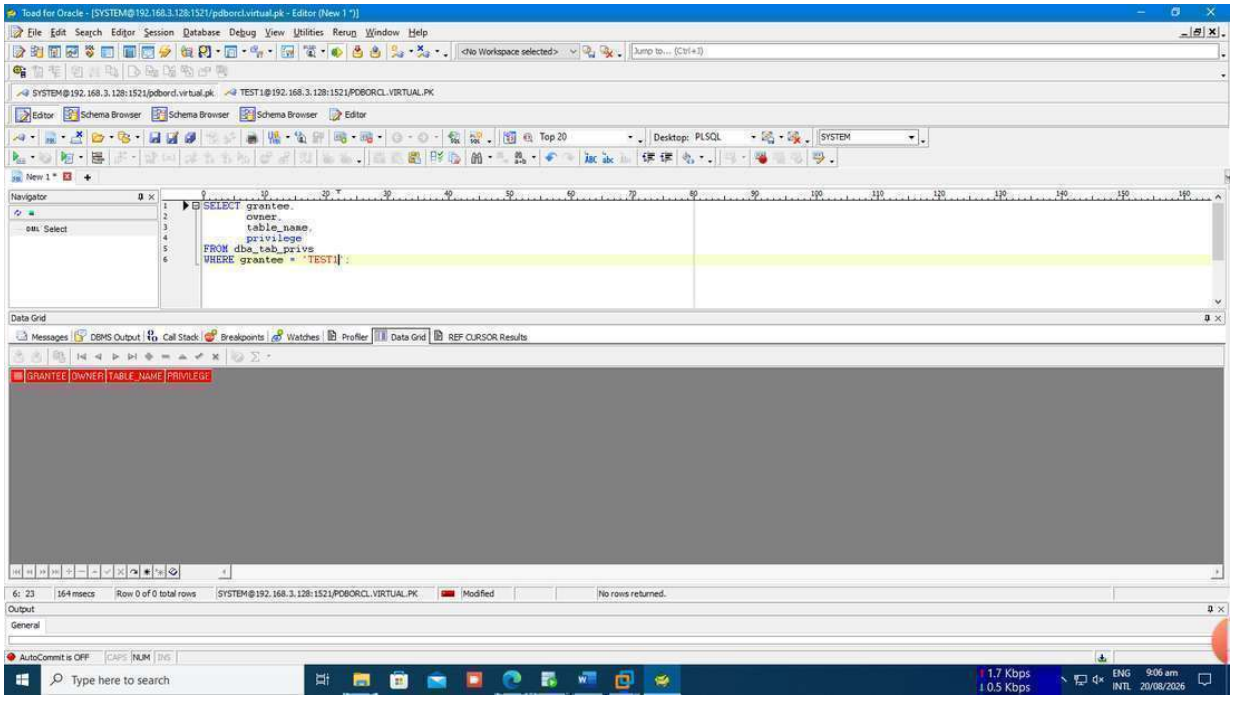
Screenshot 7.35: SELECT Privilege Revoked



Verification

```
SELECT grantee,
       owner,
       table_name,
       privilege
FROM dba_tab_privs
WHERE grantee = 'TEST_DBA';
```

Screenshot 7.36: Privilege Revocation Verification



7.4.12 Testing Access After REVOKE

After revoking the privilege, try again:

```
SELECT *
FROM boutique.products;
```

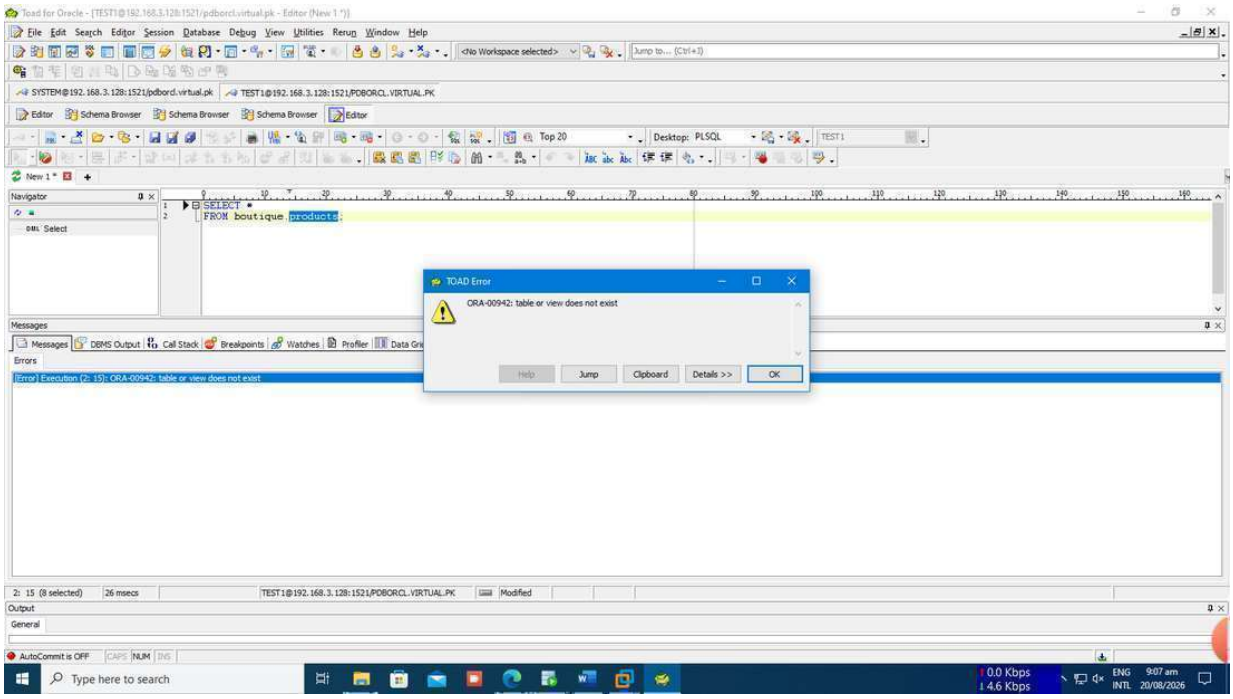
Expected Result

Oracle should return an insufficient privileges error because the `SELECT` privilege has been removed.

Explanation

This verifies that the `REVOKE` command successfully removed the user's access.

Screenshot 7.37: Access Denied After REVOKE



7.4.13 Password Management

Database administrators should follow appropriate password security practices.

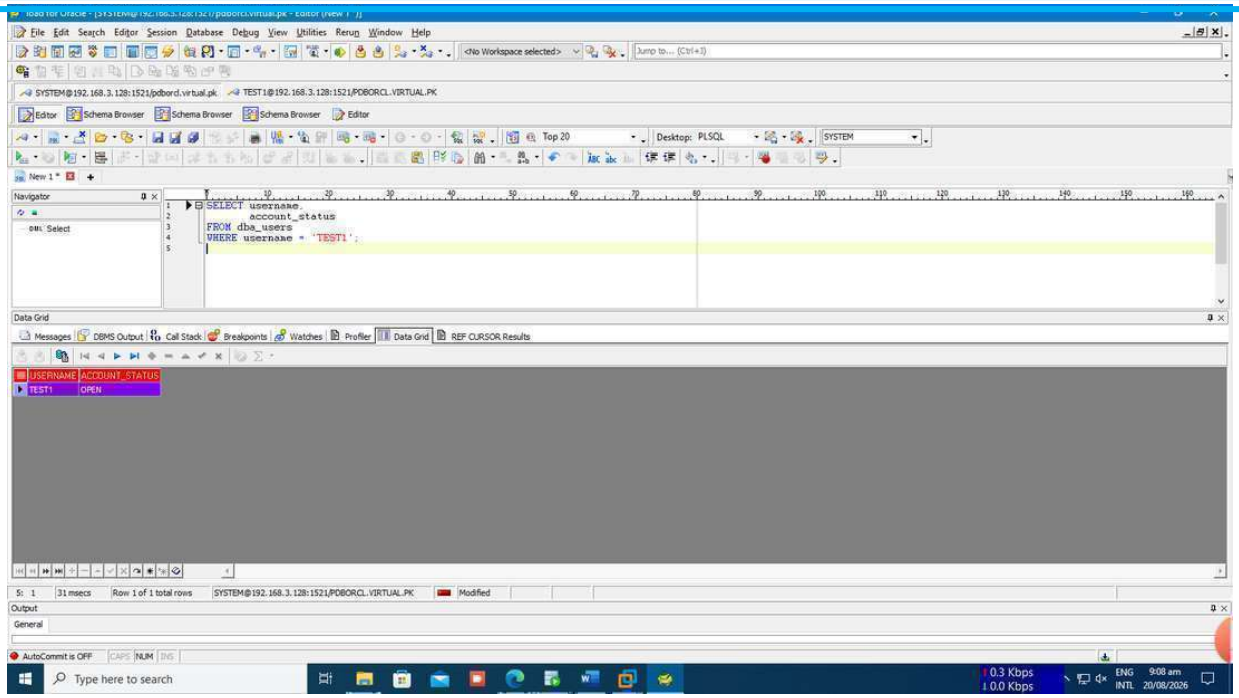
Important practices include:

- Use strong passwords.
- Do not share database passwords.
- Avoid simple passwords.
- Disable or remove unused accounts when appropriate.
- Follow organizational password policies.

Check Account Status

```
SELECT username,
       account_status
FROM dba_users
WHERE username = 'TEST1';
```

Screenshot 7.38: User Account Status



7.4.14 Principle of Least Privilege

The **Principle of Least Privilege** means that a database user should receive only the privileges required to perform their assigned work.

For example, if a user only needs to read product information, the following privilege is sufficient:

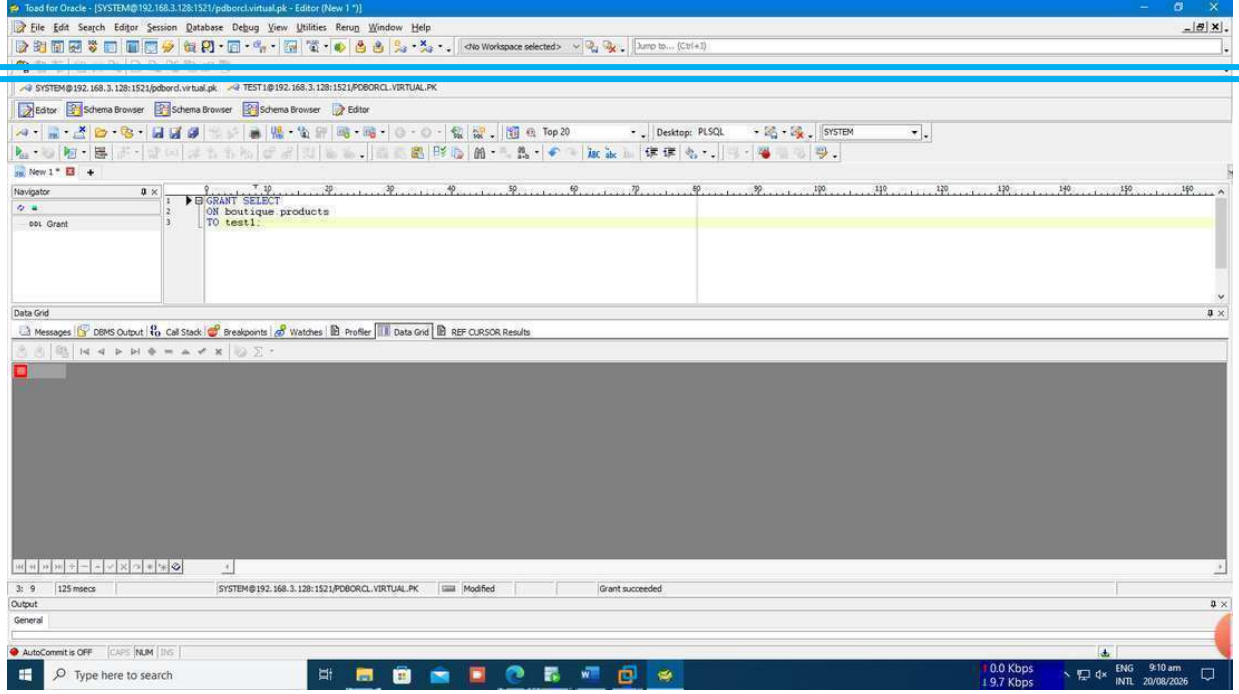
```
GRANT SELECT
ON boutique.products
TO test1;
```

There is no need to provide unnecessary administrative privileges.

Explanation

Following the principle of least privilege reduces the risk of unauthorized access and accidental modification of database information.

Screenshot 7.39: Least Privilege Demonstration



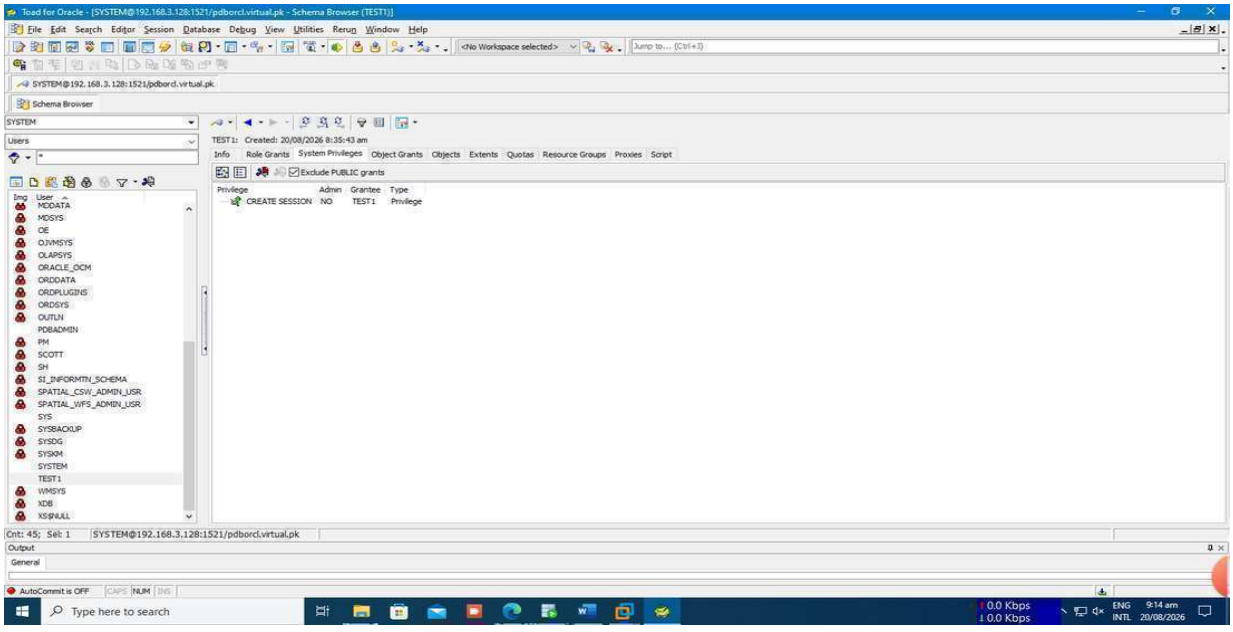
7.4.15 Viewing User Privileges in TOAD

The same user and privilege information can also be viewed through TOAD.

Procedure

1. Open TOAD.
2. Connect using an administrative account.
3. Open the **Schema Browser**.
4. Navigate to the user/security section.
5. Select TEST_DBA.
6. View the user's privileges and related information.

Screenshot 7.40: TEST1 User in TOAD



7.4.16 Removing the Test User

After completing the security demonstration, the temporary test user can be removed.

Command

```
DROP USER test1 CASCADE;
```

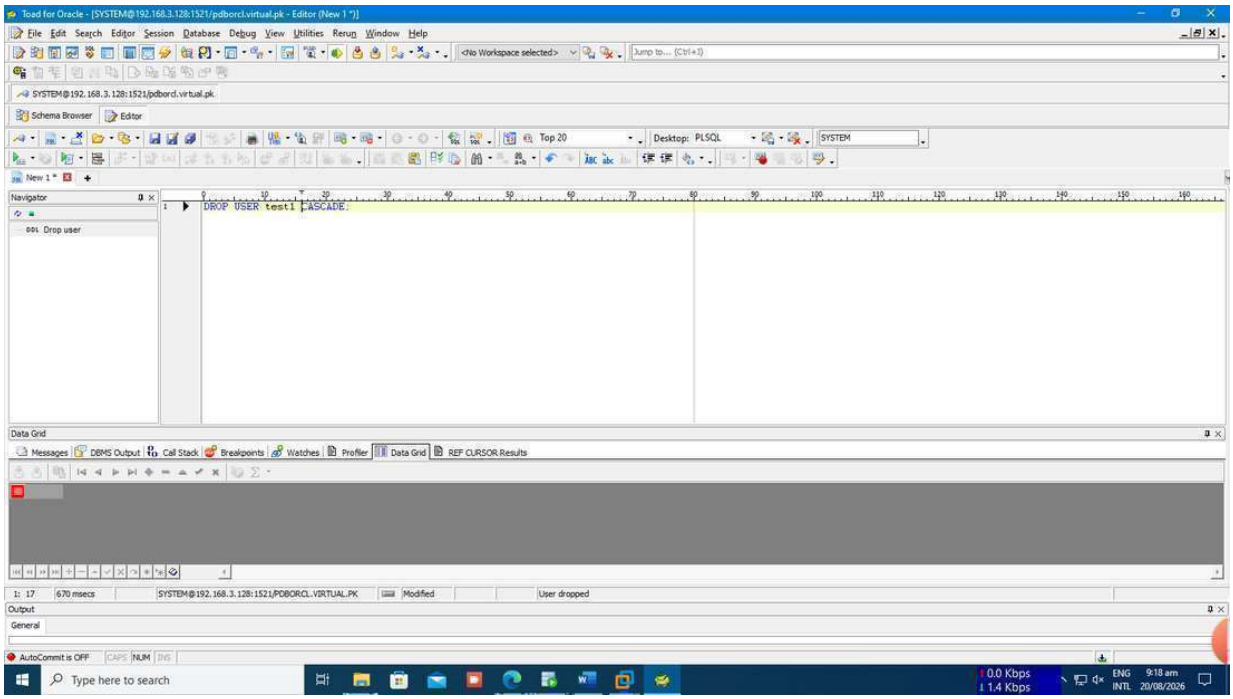
Explanation

The `DROP USER` command removes the `TEST1` account.

The `CASCADE` option removes objects owned by that user, if any exist.

Important: Only use `CASCADE` for this temporary test account. **Do not run this command against the `BOUTIQUE` project user.**

Screenshot 7.41: TEST1 User Removed



7.4.17 Security Verification

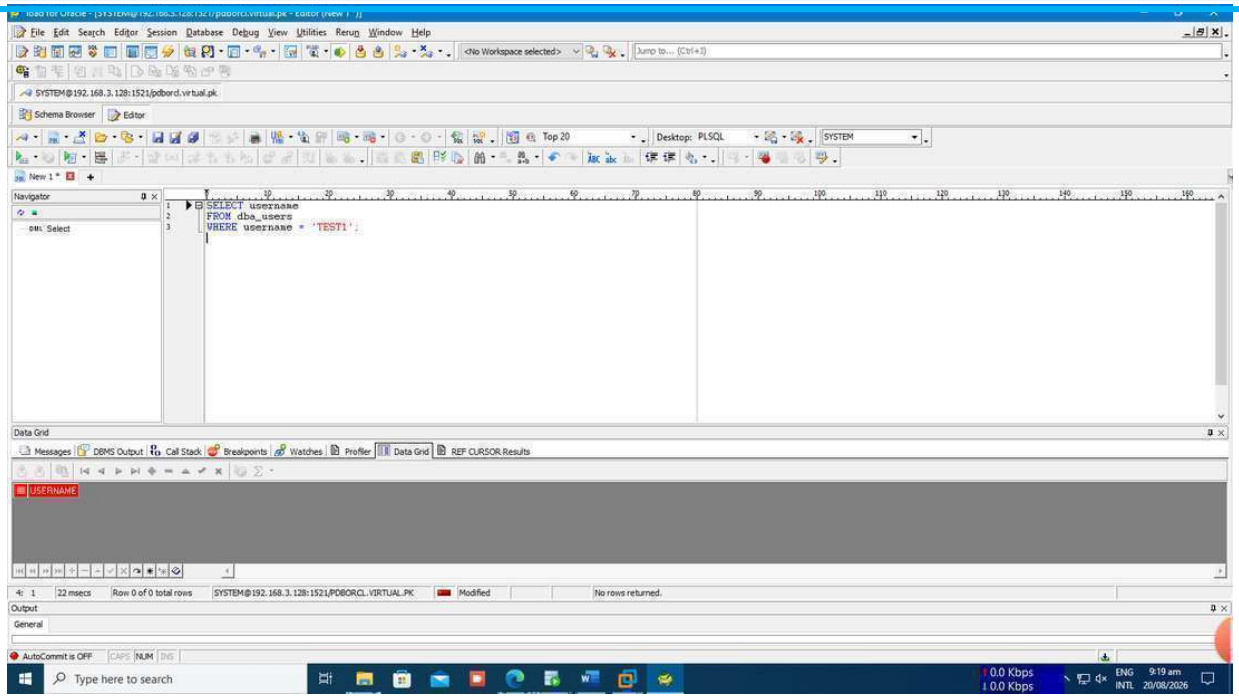
Finally, verify that the test account has been removed:

```
SELECT username
FROM dba_users
WHERE username = 'TEST1';
```

Expected Result

No row should be returned if the user was successfully removed.

Screenshot 7.42: Final Security Verification



7.4.18 Important Notes

1. Administrative commands should be executed using an appropriate administrative account.
2. Do not give unnecessary privileges to normal users.
3. The BOUTIQUE project user should not be deleted.
4. TEST_DBA is only a temporary user for demonstrating security administration.
5. Passwords used in the report should be demonstration passwords only.
6. Real production passwords must never be included in the report.
7. Privileges should be granted according to the user's actual requirements.

7.4.19 Practical Outcome

After completing this practical, the following activities were successfully understood and demonstrated:

- Database user management
- Database roles
- System privileges
- Object privileges
- User creation
- GRANT command
- REVOKE command
- Privilege verification
- Unauthorized access testing
- Password/account management concepts
- Principle of Least Privilege
- Basic security administration through TOAD **Screenshot 7.43: Final Database Security Administration**

7.6 Conclusion

In Week No. 5, Oracle Database Administration concepts and practical activities were studied using TOAD and Oracle database administration tools.

The first practical focused on **Database Links**, which demonstrated how a database can access data from another remote Oracle database.

The second practical focused on **Backup and Recovery**, including different backup types, RMAN, incremental backups, control file backups, archived redo log backups, restore and recovery concepts, and Oracle RAC.

The third practical covered **Database Monitoring and Performance Tuning**, including session monitoring, database status, instance status, tablespace monitoring, execution plans, indexes, and performance analysis concepts.

The fourth practical covered **Database Security and User/Privilege Management**, including users, roles, system privileges, object privileges, `GRANT`, `REVOKE`, password management, and the principle of least privilege.

These activities provided practical knowledge of important Oracle DBA responsibilities and prepared the database environment for real-world administration and maintenance tasks.

Practical No. 12 — Data Guard , Synomus and Auditing

8.7. Oracle Data Guard

Oracle Data Guard is an Oracle Database feature used to provide **high availability, data protection, and disaster recovery**. It maintains one or more standby databases for a primary database. The changes made in the primary database are transferred to the standby database through redo data.

The primary database is the main database where normal database operations are performed. A standby database acts as a backup and can be used if the primary database becomes unavailable.

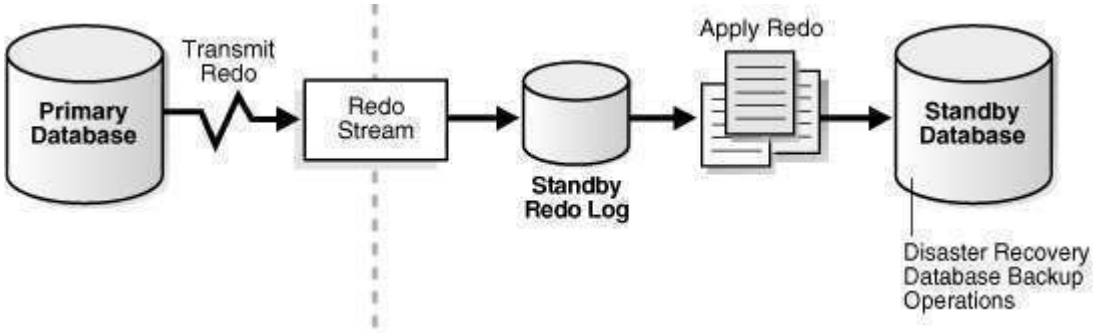
Main Components

- **Primary Database:** The main production database.
- **Standby Database:** A copy of the primary database maintained for disaster recovery.
- **Redo Data:** Contains information about changes made in the database.
- **Data Guard Services:** Manage the transfer and application of redo data between primary and standby databases.

Benefits of Data Guard

- Provides disaster recovery.
- Improves database availability.
- Protects against data loss.
- Supports planned and unplanned database failover.
- Helps maintain a standby copy of the production database.

Figure: Oracle Data Guard Primary and Standby Database Architecture



8.8. Synonyms

A synonym is an alternative name or alias for a database object such as a **table, view, sequence, procedure, or another database object**. Synonyms make it easier for users and applications to access database objects without specifying the complete object name or owner.

For example, if a table belongs to another schema, a synonym can be created so that the user can access the table using a simpler name.

Types of Synonyms

There are mainly two types of synonyms:

1. **Private Synonym** – Available only to the user who creates it.
2. **Public Synonym** – Available to all database users who have the required privileges.

Example

CREATE SYNONYM emp_syn

FOR hr.employees;

After creating the synonym, the user can access the object using:

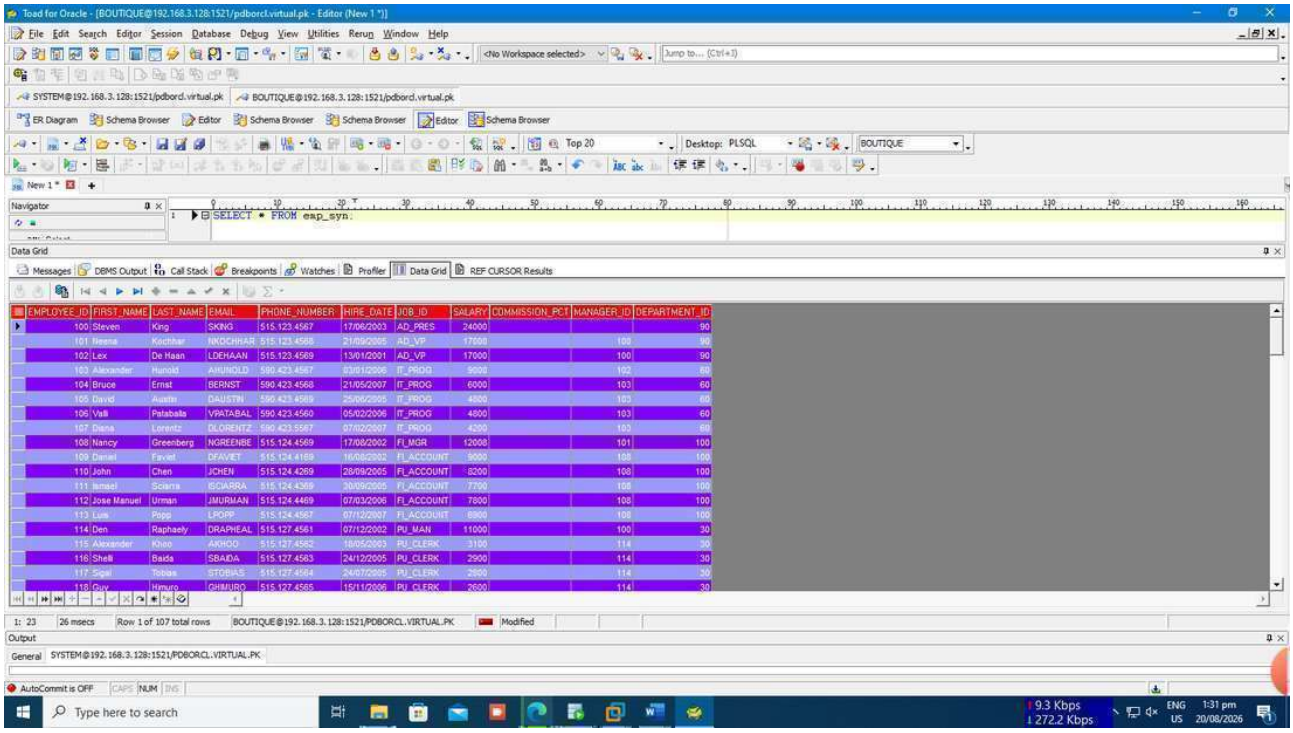
SELECT * FROM emp_syn;

Instead of:

SELECT * FROM hr.employees;

Synonyms are useful for simplifying SQL statements, hiding the owner/schema name, and providing a convenient way to access database objects.

Figure: Synonym Creation and Execution



8.9. Database Auditing

Database auditing is the process of **monitoring and recording activities performed within a database**. It helps database administrators identify who accessed the database, what actions were performed, and when those actions occurred.

Auditing is an important part of database security because it provides a record of user activities and helps detect unauthorized or suspicious operations.

Common Activities That Can Be Audited

- User logins and logouts.
- Data modification activities.
- INSERT, UPDATE, and DELETE operations.
- CREATE, ALTER, and DROP operations.
- Access to important database objects.
- Security-related activities.

Importance of Auditing

Database auditing helps in:

- Improving database security.
- Monitoring user activities.
- Detecting unauthorized access.

- Maintaining accountability.
- Supporting troubleshooting and investigation.
- Meeting security and compliance requirements.

A DBA can configure auditing according to the organization's security requirements and then review the generated audit information.

Figure: Database Auditing Configuration

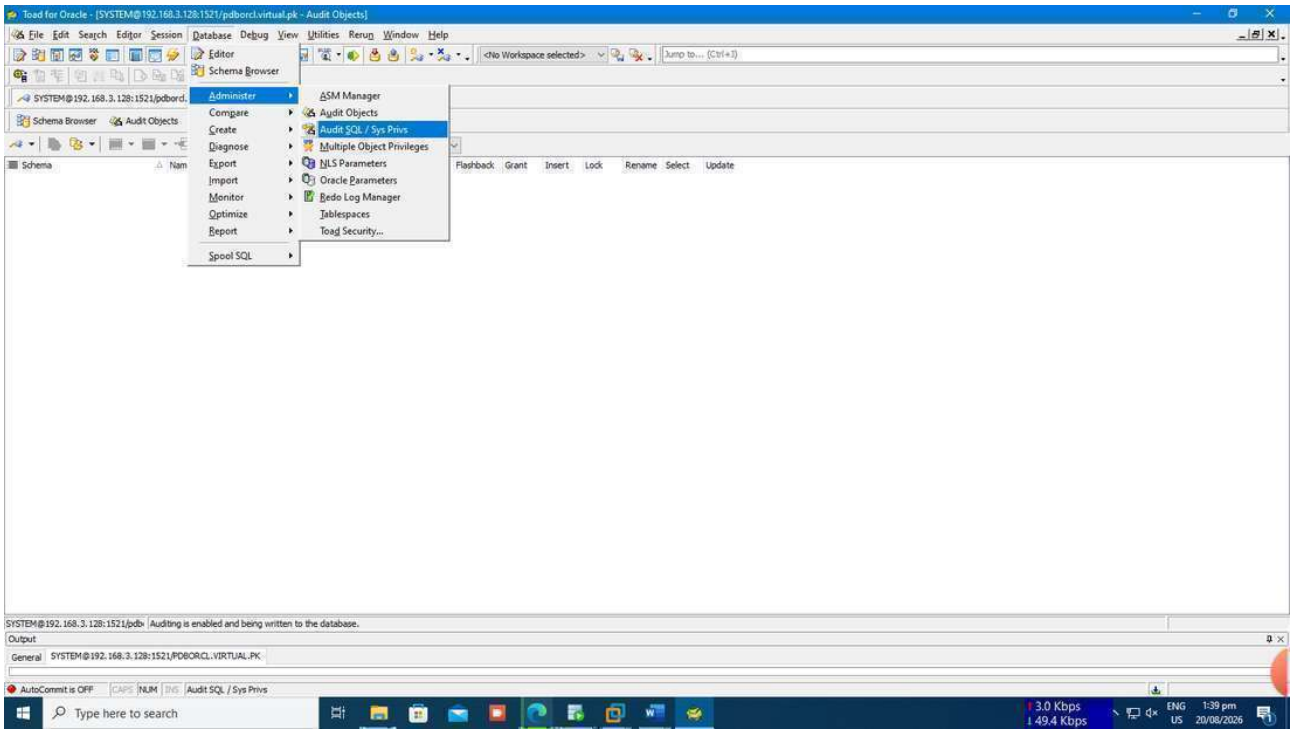


Figure: Viewing Database Audit Records

